LAPORAN MILESTONE 2 TUGAS BESAR

IF2224 TEORI BAHASA FORMAL DAN AUTOMATA

Syntax Analysis

Disusun oleh:

JBC - J1B2C2

Samuelson D. Tanuraharja	13524001
Aufa Rienaldifaza Ahmad	13524027
Niko Samuel Simanjuntak	13524029
Aurelia Jennifer Gunawan	13524089
Reinsen Silitonga	13524093

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

BAB 1
TEORI SINGKAT

1.1. Grammar

Grammar merupakan himpunan aturan produksi yang bertujuan untuk menghasilkan sebuah *string* yang valid pada suatu bahasa. *Grammar* diperlukan agar sekumpulan *string* pada suatu bahasa formal dapat menghasilkan suatu makna yang valid. Secara formal, grammar dinyatakan dengan

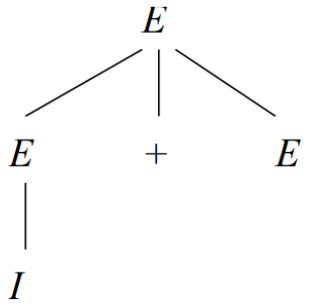
$$G = (N, T, P, S)$$

N merupakan himpunan variabel nonterminal, yaitu simbol yang dapat diturunkan menjadi simbol lain. T merupakan himpunan simbol terminal, yaitu simbol yang dapat membentuk suatu *string* yang bermakna. P merupakan himpunan aturan produksi, yaitu sekumpulan aturan yang mengubah variabel nonterminal menjadi variabel nonterminal lain atau simbol terminal. S merupakan himpunan simbol nonterminal yang memulai proses penurunan (*derivation*). Suatu *grammar* yang memiliki himpunan aturan produksi yang bertujuan untuk mendefinisikan suatu bahasa dari seluruh *string* yang mungkin diturunkan disebut dengan *Context-Free Grammar* (CFG).

Grammar dapat diturunkan dengan dua metode, yaitu *leftmost derivation* dan *rightmost derivation*. Sesuai dengan namanya, *leftmost derivation* memprioritaskan penurunan variabel nonterminal paling kiri, sedangkan *rightmost derivation* memprioritaskan penurunan variabel paling kanan. Pemilihan metode ini berpengaruh terhadap proses *parsing* yang akan digunakan. Suatu *grammar* yang diturunkan menjadi sebuah *string* yang valid akan berbentuk sebuah pohon yang disebut dengan *parse tree.*

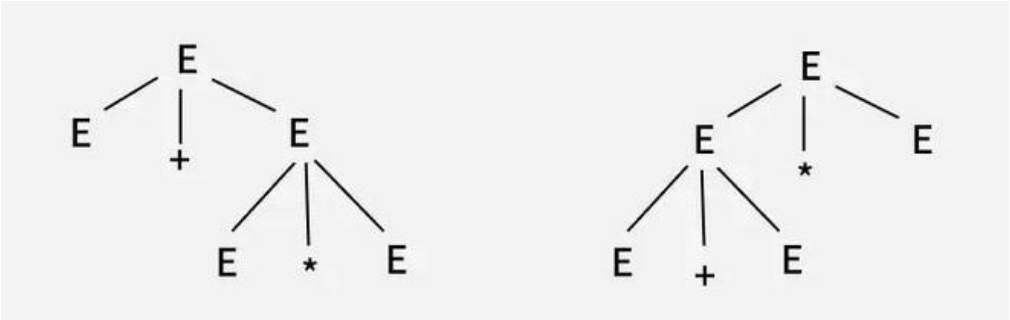
1.2.Parse Tree

Suatu *tree* merupakan *parse tree* dari sebuah CFG G = (V, T, P, S) jika seluruh *node* dalam *tree* dilabelkan berdasarkan variabel dalam V, seluruh *node* daun dilabelkan sebagai simbol dalam $V \cup T \cup \{\epsilon\}$, dan seluruh *node* berlabel A dengan *node* anaknya berlabel $X1, X2, \ldots, X_k$ merupakan $A \rightarrow X1X2\ldots X_k \in P$. Misalkan sebuah *grammar* sederhana dengan aturan produksi $P = \{E \rightarrow I, E \rightarrow E + E, E \rightarrow E * E, E \rightarrow (E)$, maka salah satu *parse tree* yang valid terdapat pada Gambar 1.2.1.



Gambar 1.2.1 *Parse tree* dari grammar dengan aturan produksi P

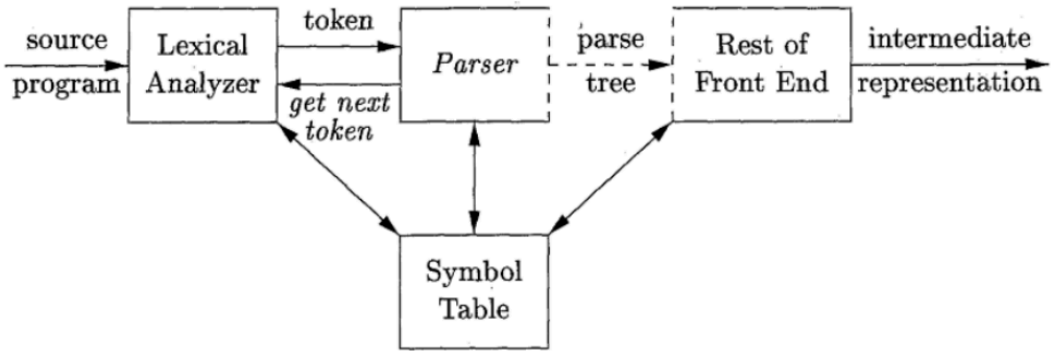
Bahasa dapat dinyatakan ambigu jika terdapat lebih dari satu susunan *parse tree* untuk suatu *string* valid. Ambiguitas ini dapat dibentuk dengan mengonstruksi suatu *parse tree* menggunakan metode yang sama, yaitu *leftmost derivation* atau *rightmost derivation*, dan menghasilkan sejumlah *parse tree* yang berbeda.



Gambar 1.2.2 Contoh *grammar* yang ambigu

1.3.Parser

Pada sebuah *compiler*, *parser* menerima kumpulan karakter (*strings*) token yang diberikan oleh *lexical analyzer*. Untuk setiap *string*, *parser* akan memverifikasi nama-nama token yang dapat dibentuk berdasarkan *grammar* dari suatu bahasa. Selain memverifikasi sekumpulan *token* berdasarkan *grammar* bahasa, *parser* memiliki peran untuk mengembalikan *error code* pada saat terjadi *syntax error*, juga melakukan *recovery* untuk suatu error yang rawan terjadi sehingga dapat memproses keseluruhan program. Setelah suatu program melalui proses *parsing* pada suatu *compiler*, akan terbentuk suatu *parse tree* yang akan diproses lebih lanjut pada langkah *compilation* berikutnya.



Gambar 1.3.1. Peran *Parser* dalam *Compiler*

Berdasarkan bagaimana suatu *parse tree* disusun, *Parsing* diklasifikasikan menjadi dua tipe, *top down parsing* dan *bottom up parsing*. *Top down parsing* mengkonstruksi *parse tree* pada *root node*, kemudian mengekspan *node* anakannya. *Bottom up parsing* mengkonstruksi *parse tree* dimulai dari *leaf node*, kemudian melakukan *traversal* menuju *root node*.

Pada tugas besar ini, digunakan metode *top down parsing*. Pada dasarnya *top down parsing* dapat dilakukan dengan mencari *leftmost derivation* dari suatu input. Salah satu varian dari *top down parsing* adalah *recursive descent*.

1.4.Algoritma Recursive Descent

Algoritma *recursive descent* tersusun dari himpunan prosedur rekursif untuk masing-masing variabel nonterminal. Setiap prosedur memiliki tanggung jawab untuk menge-*parse* variabel berdasarkan aturan produksi nonterminalnya. Eksekusi dimulai dari prosedur dari *start symbol* yang akan berhenti saat prosedur berhasil melakukan konstruksi dari string input.

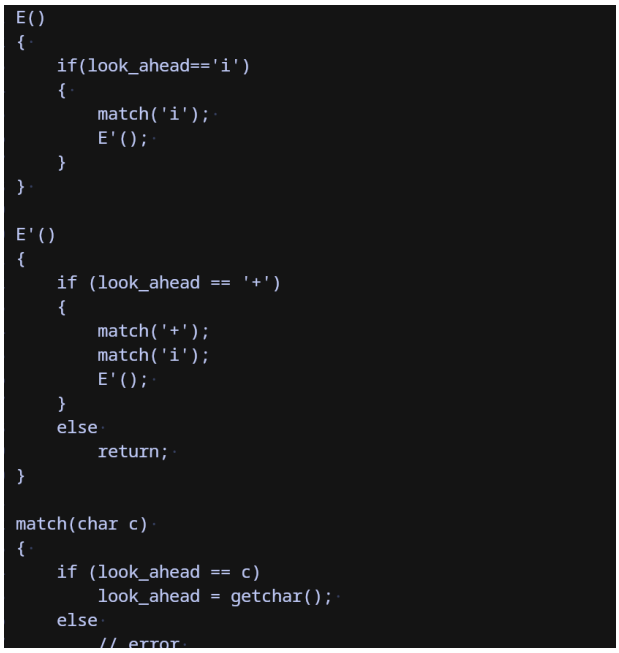
Terdapat dua batasan pada algoritma *recursive descent.* Pertama, *grammar* yang diturunkan harus merupakan *grammar* yang berasal dari bahasa yang tidak ambigu. Kedua, aturan produksi tidak boleh memiliki rekursi pada variabel paling kiri pada seluruh aturan produksi.
Misalkan diberikan sebuah grammar sebagai berikut,

$$E \rightarrow iE'$$

$$E' \rightarrow +iE' \mid \epsilon$$

Gambar 1.4.1 Contoh Aturan Produksi

aturan produksi pada Gambar 1.4.1 akan dibentuk menjadi sekumpulan prosedur yang tersusun pada Gambar 1.4.2.



```
E()
{
    if(look_ahead=='i')
    {
        match('i');
        E'();
    }
}

E'()
{
    if (look_ahead == '+')
    {
        match('+');
        match('i');
        E'();
    }
    else
        return;
}

match(char c)
{
    if (look_ahead == c)
        look_ahead = getchar();
    else
        // error
```

Gambar 1.4.2. Contoh Prosedur Recursive Descent

BAB 2
PERANCANGAN DAN IMPLEMENTASI

2.1.Teknologi yang Digunakan

Program *parser* ditulis dengan bahasa pemrograman C++ (standar C++11 atau lebih tinggi) tanpa dependensi dari pihak ketiga. Pemilihan C++ didasarkan pada kebutuhan akan performa tinggi dalam pemrosesan token dan fleksibilitas dalam pengelolaan memori secara eksplisit. Lingkungan pengembangan yang digunakan bersifat portabel, dengan dukungan *build tool* CMake versi minimal 3.16 untuk menjamin kompilasi lintas *platform*. Program bekerja dalam tiga tahap, yaitu tokenisasi, *parsing*, dan konstruksi *parse tree*. Tokenisasi dilakukan oleh *lexical analyzer* yang sebelumnya telah dikembangkan pada milestone 1. Kemudian, *parsing* dengan pendekatan *recursive descent* dilakukan untuk memvalidasi struktur *syntax* berdasarkan grammar bahasa Arion. Terakhir, *parse tree* dibangun sebagai representasi hierarkis program.

2.2.Aturan Grammar

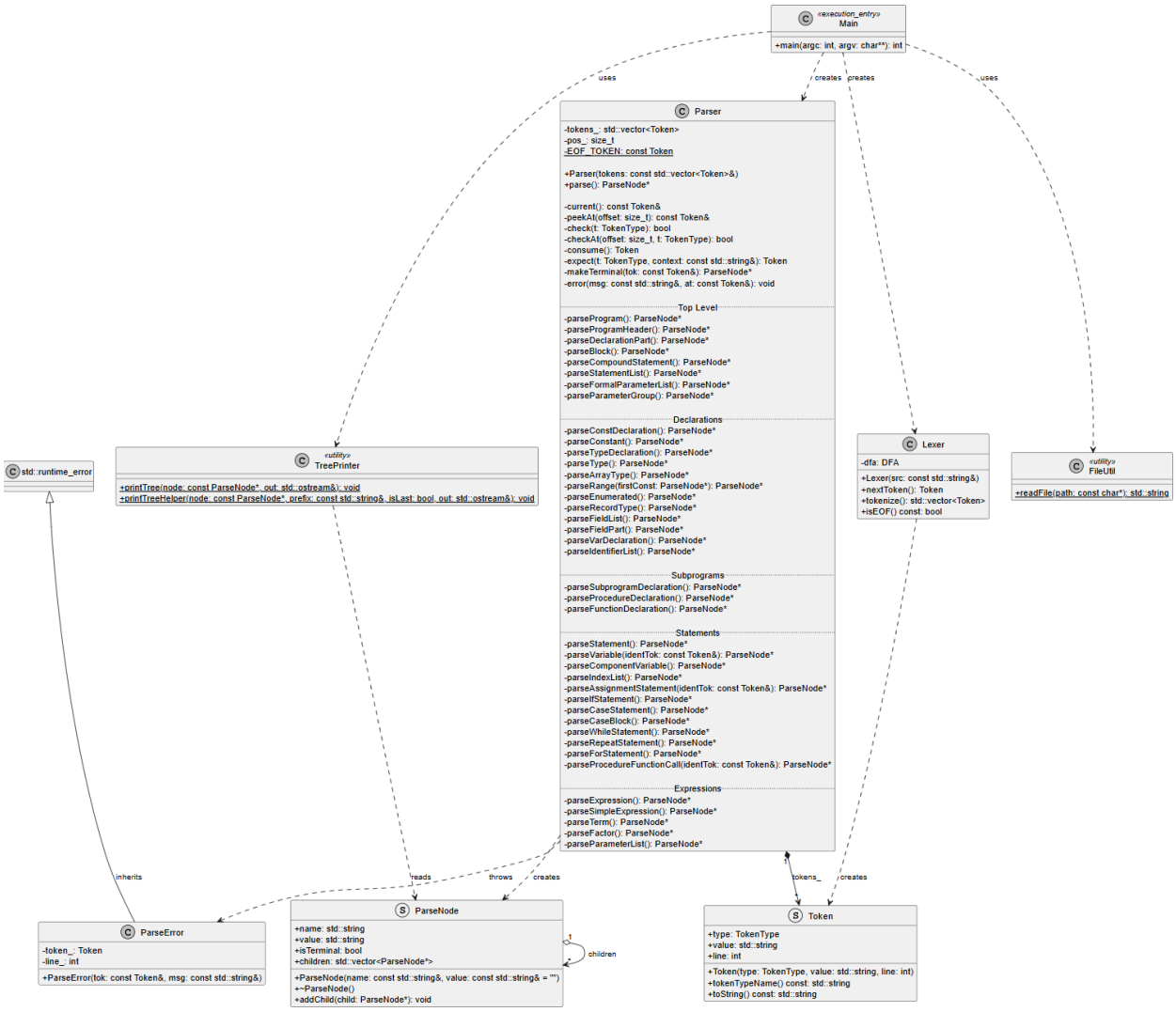
Grammar yang terdefinisi dalam program *syntax analyzer* ini memiliki anggota komponen seperti yang tertera pada lampiran.

2.3.Struktur Program

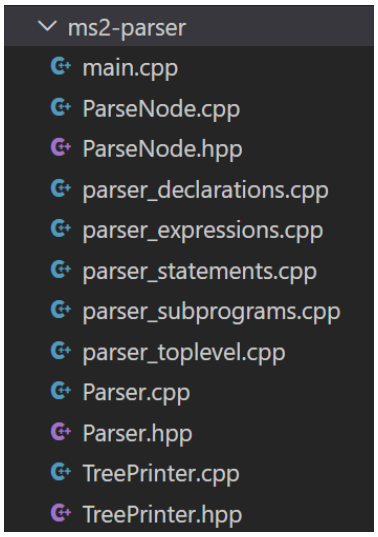
Arsitektur program *parser* dirancang secara modular dengan pemisahan tanggung jawab yang jelas antar komponen. Komponen utama terdiri dari ParseNode, Parser, TreePrinter, dan ParseError. Kelas ParseNode merepresentasikan simpul pada *parse tree* dengan atribut nama, nilai, tipe, dan pointer ke children nodes. Kelas Parser menjadi komponen sentral yang mengelola proses *parsing* dengan menyimpan token dalam sebuah *vector of Token* sebagai pointer posisi. Setiap nonterminal dalam grammar direpresentasikan sebagai *method parsing* yang terorganisasi dalam beberapa file sesuai kategorinya, seperti top-level, deklarasi, statement, ekspresi, dan subprogram.

Alur data dimulai dari lexer yang menghasilkan token, kemudian diproses oleh Parser melalui pemanggilan parse() yang memulai *recursive descent* dari *start symbol*. Parser mengkonsumsi token secara sekuensial dna membangun *parse tree* secara rekursif *depth-first* dengan membentuk ParseNode dan relasi parent-child. Hasil akhir berupa *root node* kemudian diproses oleh TreePrinter.

Penanganan error dilakukan melalui dua buah mekanisme, yaitu validasi secara proaktif dengan *lookahead* untuk menentukan *rule* yang sesuai, dan validasi secara reaktif melalui expect() yang melempar ParseError ketika terjadi ketidakcocokan token. Exception ini ditangani di level utama program untuk memberikan pesan *error* dan menghentikan eksekusi. Pada implementasi *error handling*, pesan error akan disimpan dalam sebuah list (vector) yang kemudian akan dikembalikan seluruhnya pada saat eksekusi program selesai.



Gambar 2.3.1 Diagram Kelas Program Parser



Gambar 2.3.2 Struktur Direktori Program Parser

2.4.Fungsi/Kelas Utama dan Implementasinya

2.4.1. File ParseNode.hpp dan ParseNode.cpp

Tabel 2.4.1. Implementasi kelas ParseNode

Kelas/Fungsi/Prosedur	**Penjelasan**

struct ParseNode
`struct ParseNode {`
`  std::string  name;`
`  std::string  value;`
`  bool  isTerminal;`
`  std::vector<ParseNode*> children;`
`  ParseNode(const std::string& name, const std::string& value = "");`
`  ~ParseNode();  // recursively deletes all children`
`  void addChild(ParseNode* child); // silently ignores nullptr`
`};`
	Struktur data parseNode merupakan kelas *public* yang memiliki nama node, value yang dimiliki node, nilai boolean untuk node terminal, dan *vector of* parseNode yang merupakan *children*.

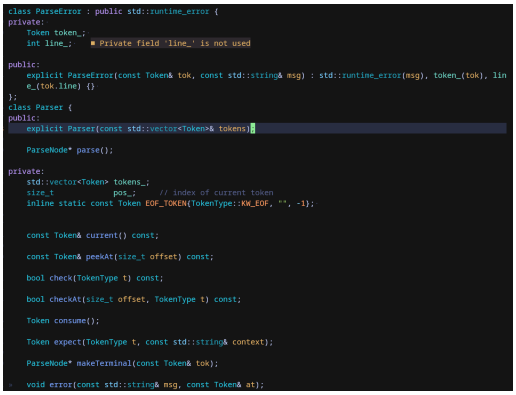
ParseNode::ParseNode(const std::string& name, const std::string& value)
`ParseNode::ParseNode(const std::string& name, const std::string& value)`
`  : name(name), value(value), isTerminal(false) {}`
	Konstruktor yang menerima nama node, dan value yang dapat berupa string kosong untuk nilai default.

ParseNode::~ParseNode()
`ParseNode::~ParseNode() {`
`  for (ParseNode* child : children) {`
`  delete child;`
`  }`
`}`
	Destructor yang melakukan penghapusan memori untuk setiap *children* yang dimiliki oleh *parent node*.

ParseNode::addChild(ParseNode* child) `void ParseNode::addChild(ParseNode* child) { if (child != nullptr) { children.push_back(child); } }`	Menambahkan child node.

2.4.2. File Parser.hpp dan Parser.cpp

Tabel 2.4.2. Implementasi kelas Parser

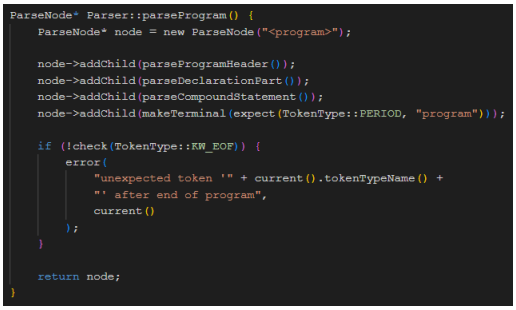
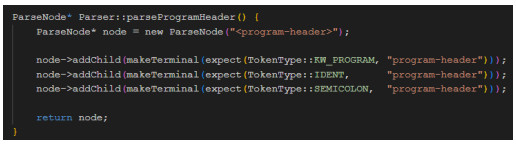
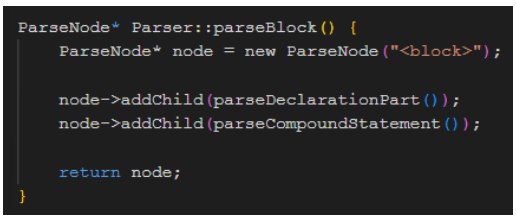
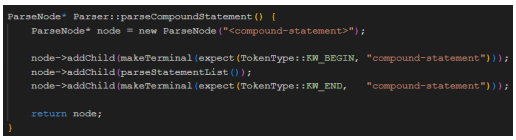
Kelas/Fungsi/Prosedur	**Penjelasan**
class Parser 	Kelas publik parse memiliki tanggung jawab untuk melakukan parsing sesuai dengan Grammar yang ditentukan yang akan didefinisikan pada masing-masing simbol terminal dan variabel nonterminal.
Parser::Parser(const std::vector<Token>& tokens) `Parser::Parser(const std::vector<Token>& tokens) : tokens_(tokens), pos_(0) {}`	Konstruktor kelas Parse.
const Token& Parser::current() const `const Token& Parser::current() const { if (pos_ >= tokens_.size()) return EOF_TOKEN; return tokens_.at(pos_); }`	Mengambil token pada posisi suatu *node*.
const Token& Parser::peekAt(size_t offset) const `const Token& Parser::peekAt(size_t offset) const { if (pos_ + offset >= tokens_.size()) return EOF_TOKEN; return tokens_.at(pos_ + offset); }`	Mengambil token pada current() + offset.

bool Parser::check(TokenType t) const `bool Parser::check(TokenType t) const { return current().type == t; }`	Mengecek tipe token sama dengan parameter t atau tidak.
Token Parser::consume() `Token Parser::consume() { if (pos_ >= tokens_.size()) return EOF_TOKEN; return tokens_[pos_++]; }`	Menambah posisi pos_ untuk mendapatkan token selanjutnya. Apabila mendapatkan end of stream, maka mengembalikan EOF_TOKEN.
Token Parser::expect(TokenType t, const std::string& context) `Token Parser::expect(TokenType t, const std::string& context) { if (!check(t)) { error("in " + context + ": expected " + Token(t, "", -1).tokenTypeName() + ", got " + current().tokenTypeName(), current()); synchronize(); if (!check(t)) { return Token(t, "<missing>", current().line); } } return consume(); }`	Melakukan pengecekan token yang sedang dibaca. Apabila check mengembalikan false, maka memanggil fungsi error dan melakukan sinkronisasi dengan token valid terdekat agar tidak terjadi *cascading failure*.
ParseNode* Parser::makeTerminal(const Token& tok) `ParseNode* Parser::makeTerminal(const Token& tok) { ParseNode* node = new ParseNode(tok.tokenTypeName(), tok.value); node->isTerminal = true; return node; }`	Membuat suatu node terminal sesuai dengan simbol-simbol terminal.
void Parser::error(const std::string& msg, const Token& at) `void Parser::error(const std::string& msg, const Token& at) { throw ParseError(at, "Syntax error at line " + std::to_string(at.line) + ": " + msg + " (got '" + at.value + "')"); }`	Mengembalikan message error sesuai dengan posisi token.

2.4.3. File parser_toplevel.cpp

Tabel 2.4.3. Implementasi kelas parse_toplevel

Kelas/Fungsi/Prosedur	**Penjelasan**
ParseNode* Parser::parseProgram()	Fungsi root yang membangun node paling atas dari parse tree (<program>). Memanggil parseProgramHeader(), parseDeclarationPart(), dan

	parseCompoundStatement() secara berurutan, lalu mengonsumsi token period sebagai penutup program. Setelah periode, fungsi juga memverifikasi bahwa tidak ada token tersisa di luar batas akhir program.
ParseNode* Parser::parseProgramHeader() 	Memparsing kepala program yang terdiri dari tiga token berurutan: programsy, ident (nama program), dan semicolon. Menghasilkan node <program-header> dengan ketiga token tersebut sebagai terminal children.
ParseNode* Parser::parseDeclarationPart() 	Memparsing seluruh blok deklarasi sebelum begin yang terdiri dari empat seksi opsional dengan urutan yang ketat: const, type, var, dan subprogram (prosedur/fungsi). Setiap seksi diparse dalam loop selama token pembuka seksi tersebut (constsy, typesy, varsy, proceduresy/functionsy) masih ditemukan.
ParseNode* Parser::parseBlock() 	Memparsing satu blok kode yang terdiri dari declaration-part diikuti compound-statement. Fungsi ini tidak digunakan di level program utama, melainkan dipanggil secara internal oleh parseProcedureDeclaration() dan parseFunctionDeclaration() untuk memparsing tubuh prosedur dan fungsi.
ParseNode* Parser::parseCompoundStatement() 	Memparsing blok pernyataan yang diawali beginsy, diisi oleh parseStatementList(), dan ditutup endsy. Menghasilkan node <compound-statement> dengan ketiga bagian tersebut sebagai children.

ParseNode* Parser::parseStatementList()
`ParseNode* Parser::parseStatementList() {`
`  ParseNode* node = new ParseNode("<statement-list>");`
`  node->addChild(parseStatement());`
`  while (check(TokenType::SEMICOLON)) {`
`  node->addChild(makeTerminal(consume()));`
`  node->addChild(parseStatement());`
`  }`
`  return node;`
`}`
	Memparsing daftar statement yang dipisahkan oleh semicolon sesuai aturan produksi statement (semicolon statement)*. Token semicolon selalu ditambahkan ke tree sebagai terminal node, sementara statement kosong (yang menghasilkan nullptr dari parseStatement()) diabaikan secara otomatis oleh guard addChild().

ParseNode* Parser::parseFormalParameterList()
`ParseNode* Parser::parseFormalParameterList() {`
`  ParseNode* node = new ParseNode("<formal-parameter-list>");`
`  node->addChild(makeTerminal(expect(TokenType::LPAR, "formal-parameter-list")));`
`  node->addChild(parseParameterGroup());`
`  while (check(TokenType::SEMICOLON)) {`
`  node->addChild(makeTerminal(consume()));`
`  node->addChild(parseParameterGroup());`
`  }`
`  node->addChild(makeTerminal(expect(TokenType::RPAR, "formal-parameter-list")));`
`  return node;`
`}`
	Memparsing daftar parameter formal prosedur atau fungsi dalam format lparent parameter-group (semicolon parameter-group)* rparent. Fungsi ini hanya dipanggil ketika token lparent terdeteksi setelah nama prosedur atau fungsi.

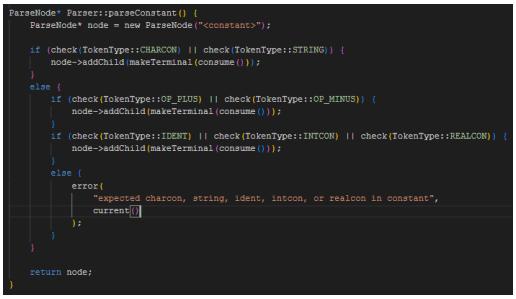
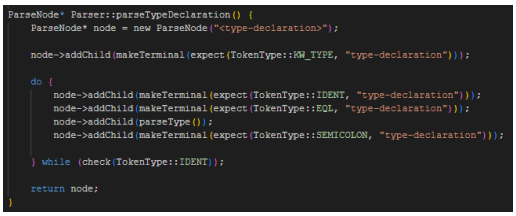
ParseNode* Parser::parseParameterGroup()
`ParseNode* Parser::parseParameterGroup() {`
`  ParseNode* node = new ParseNode("<parameter-group>");`
`  node->addChild(parseIdentifierList());`
`  node->addChild(makeTerminal(expect(TokenType::COLON, "parameter-group")));`
`  if (check(TokenType::KW_ARRAY)) {`
`  node->addChild(parseArrayType());`
`  } else if (check(TokenType::IDENT)) {`
`  node->addChild(makeTerminal(consume()));`
`  } else {`
`  error(`
`  "expected type name or 'array' in parameter-group, got '" +`
`  current().tokenTypeName() + "'",`
`  current()`
`  );`
`  }`
`  return node;`
`}`
	Memparsing satu grup parameter yang terdiri dari identifier-list, colon, dan tipe parameter. Tipe parameter dibatasi hanya pada nama tipe (ident) atau array-type, sesuai aturan produksi <parameter-group>.

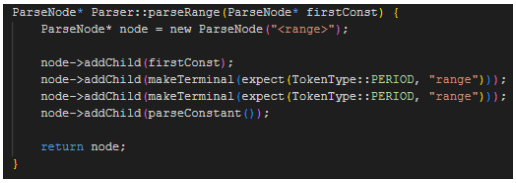
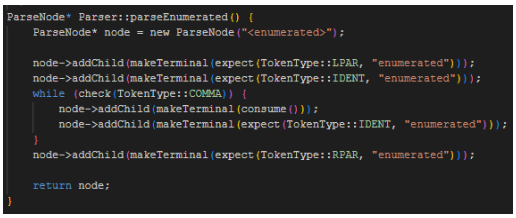
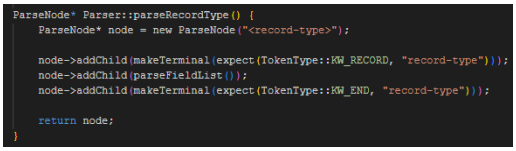
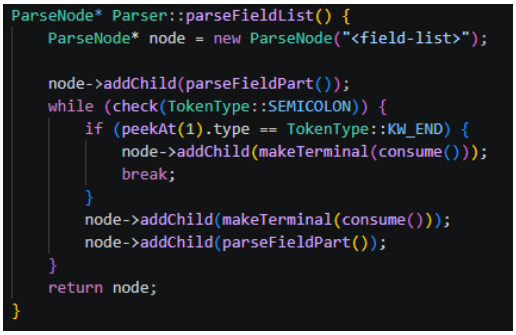
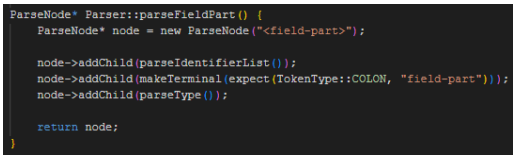
2.4.4. File parser_declarations.cpp

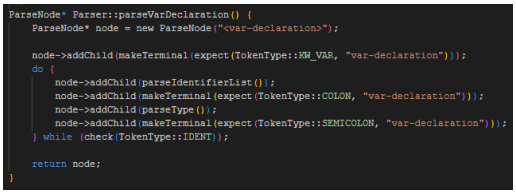
Tabel 2.4.4. Implementasi kelas parser_declarations

Kelas/Fungsi/Prosedur	**Penjelasan**

ParseNode* Parser::parseConstDeclaration()
`ParseNode* Parser::parseConstDeclaration() {`
`  ParseNode* node = new ParseNode("<const-declaration>");`
`  node->addChild(makeTerminal(expect(TokenType::KW_CONST, "const-declaration")));`
`  do {`
`  node->addChild(makeTerminal(expect(TokenType::IDENT, "const-declaration")));`
`  node->addChild(makeTerminal(expect(TokenType::EQL, "const-declaration")));`
`  node->addChild(parseConstant());`
`  node->addChild(makeTerminal(expect(TokenType::SEMICOLON, "const-declaration")));`
`  } while (check(TokenType::IDENT));`
`  return node;`
`}`
	Menangani deklarasi satu atau lebih konstanta yang diawali dengan kata kunci const. Fungsi ini melakukan pengulangan untuk setiap pasang pengenal, tanda sama dengan (=), nilai konstanta, dan titik koma.

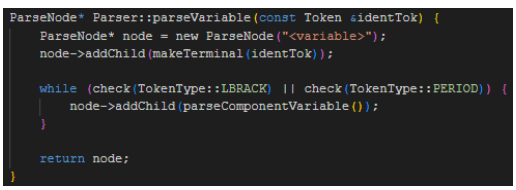
ParseNode* Parser::parseConstant()  	Memproses nilai konstanta yang dapat berupa literal karakter, literal string, atau kombinasi tanda operasional (+ atau -) dengan angka atau pengenal.
ParseNode* Parser::parseTypeDeclaration()  	Menangani blok definisi tipe data baru yang diawali dengan kata kunci type. Fungsi ini memproses satu atau lebih pasang nama tipe, tanda sama dengan (=), definisi tipe, dan titik koma.
ParseNode* Parser::parseType()  	Berfungsi sebagai pemilih (*dispatcher*) untuk menentukan jenis tipe data yang sedang diproses, baik tipe sederhana (pengenal), array, enumerasi, rekaman, atau rentang (*range*).
ParseNode* Parser::parseArrayType()  	Memproses definisi tipe data array, mulai dari kata kunci array, penentuan rentang indeks di dalam kurung siku, kata kunci of, hingga tipe data elemen array tersebut.

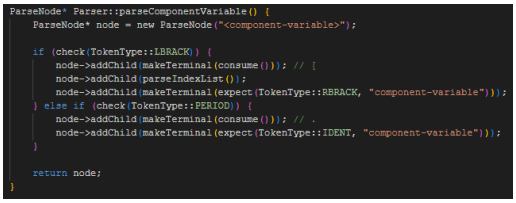
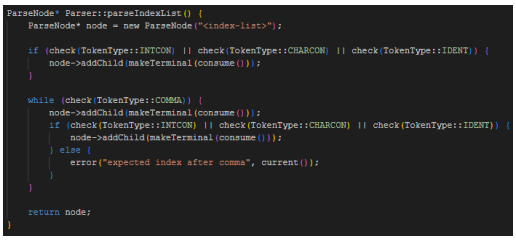
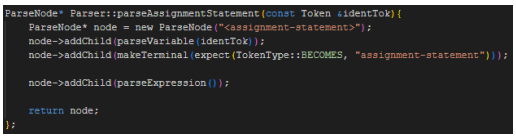
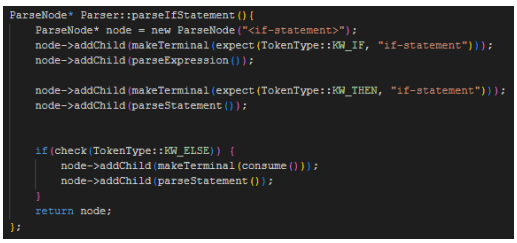
ParseNode* Parser::parseRange(ParseNode* firstExpr)  	Menangani pembentukan node rentang (misal 1..10) dengan mengonsumsi dua token titik (..) dan nilai konstanta batas atas. Parameter firstConst adalah nilai batas bawah yang sudah diproses sebelumnya.
ParseNode* Parser::parseEnumerated()  	Memproses tipe data enumerasi yang terdiri dari daftar pengenal di dalam tanda kurung yang dipisahkan oleh tanda koma.
ParseNode* Parser::parseRecordType()  	Menangani definisi tipe data rekaman (*record*) yang diapit oleh kata kunci record dan end, serta memanggil fungsi pemroses daftar atribut di dalamnya.
ParseNode* Parser::parseFieldList()  	Memproses daftar atribut atau *field* di dalam sebuah *record*. Fungsi ini menangani pemisahan antar atribut menggunakan tanda titik koma.
ParseNode* Parser::parseFieldPart()  	Menangani satu bagian atribut rekaman yang terdiri dari daftar pengenal (*identifier list*), tanda titik dua, dan tipe datanya.
ParseNode* Parser::parseVarDeclaration()	Memproses deklarasi satu atau lebih variabel yang diawali dengan kata kunci var, yang mencakup daftar nama variabel, tanda titik dua, tipe data, dan diakhiri

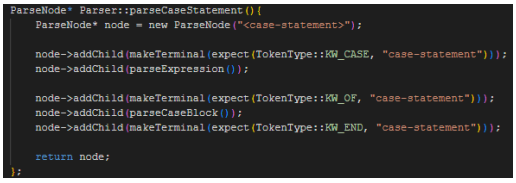
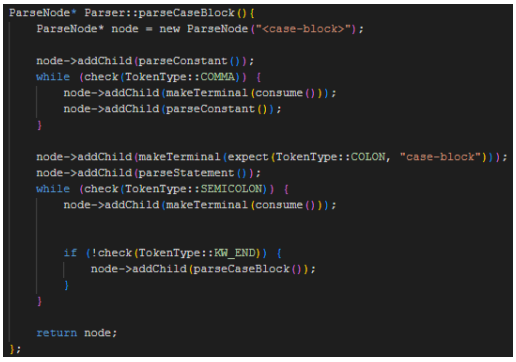
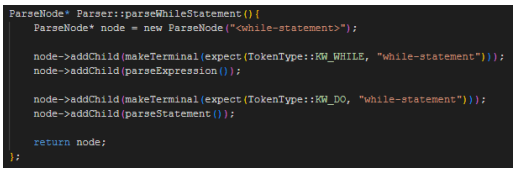
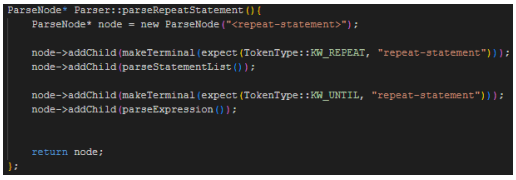
	titik koma.
ParseNode* Parser::parseIdentifierList() 	Menangani daftar satu atau lebih nama pengenal (*identifier*) yang dipisahkan oleh tanda koma.

2.4.5. File parser_statements.cpp

Tabel 2.4.5. Implementasi kelas parser_statements

Kelas/Fungsi/Prosedur	**Penjelasan**
ParseNode* Parser::parseStatement() 	Method dispatcher yang menentukan jenis statement berdasarkan leading token. Mendelegasikan parsing ke method spesifik seperti parseIfStatement(), parseWhileStatement(), dll. Mengembalikan nullptr jika tidak ada statement.
ParseNode* Parser::parseVariable(const Token& identTok) 	Melakukan *parsing* untuk *variable access* yang dapat berupa *identifier* sederhana atau dengan *component variable* (*array indexing* atau *record field access*). Token *identifier* telah dikonsumsi sebelumnya dan diteruskan sebagai parameter.

ParseNode* Parser::parseComponentVariable() 	Melakukan *parsing* untuk akses komponen *variable*: [index-list] untuk *array* atau .ident untuk *record field*.
ParseNode* Parser::parseIndexList() 	Melakukan *parsing* untuk daftar *index* dalam *array access*, dapat berupa intcon, charcon, atau ident. Multi-dimensional *array* dipisahkan koma.
ParseNode* Parser::parseAssignmentStatement(const Token& identTok) 	Melakukan *parsing* untuk *assignment statement* variable := expression. *Identifier* token telah dikonsumsi di level atas untuk menghindari ambiguitas dengan *procedure call*.
ParseNode* Parser::parseIfStatement() 	Melakukan *parsing* untuk *conditional statement* dengan *syntax* if expression then statement [else statement].
ParseNode* Parser::parseCaseStatement()	Melakukan *parsing* untuk multi-way *branch* case expression of case-block end.

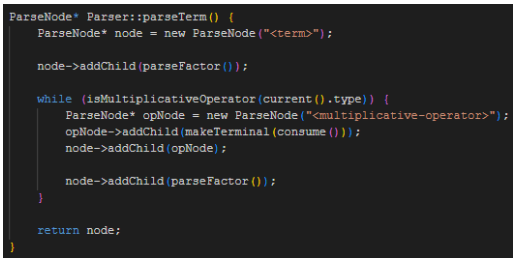
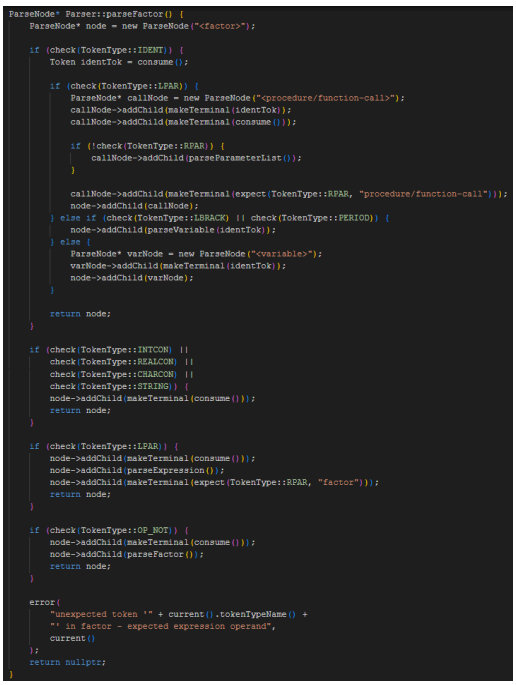
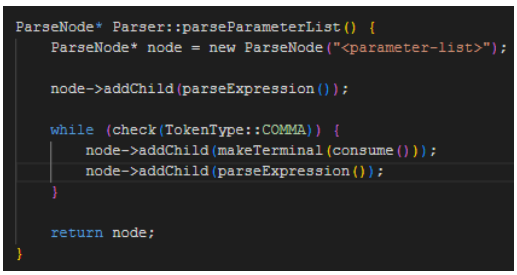
	
ParseNode* Parser::parseCaseBlock() 	Melakukan *parsing* untuk satu atau lebih *case arms* dalam bentuk constant [, constant]* : statement. Menggunakan rekursi untuk menangani *multiple arms*.
ParseNode* Parser::parseWhileStatement() 	Melakukan *parsing* untuk pre-test loop while expression do statement.
ParseNode* Parser::parseRepeatStatement() 	Melakukan *parsing* untuk post-test loop repeat statement-list until expression.
ParseNode* Parser::parseForStatement() 	Melakukan *parsing* untuk *counting loop* for ident := expression to/downto expression do statement

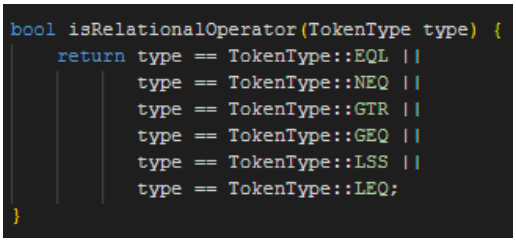
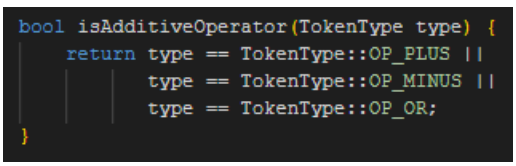
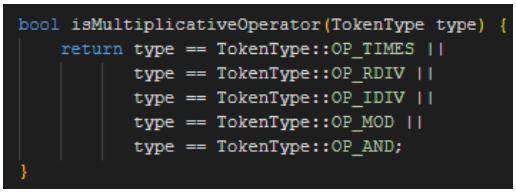
ParseNode* Parser::parseProcedureFunctionCall(const Token& identTok) `ParseNode* Parser::parseProcedureFunctionCall(const Token &identTok){ ParseNode* node = new ParseNode("<procedure/function-call>"); node->addChild(makeTerminal(identTok)); node->addChild(makeTerminal(expect(TokenType::LPAR, "procedure/function-call"))); if(!check(TokenType::RPAR)) { node->addChild(parseParameterList()); } node->addChild(makeTerminal(expect(TokenType::RPAR, "procedure/function-call"))); return node; };`	Melakukan *parsing* untuk pemanggilan *procedure/function* dengan *optional parameter list*. Token *identifier* sudah dikonsumsi untuk menghindari ambiguitas dengan *variable access*.

2.4.6. File parser_expressions.cpp

Tabel 2.4.6. Implementasi kelas parser_expressions

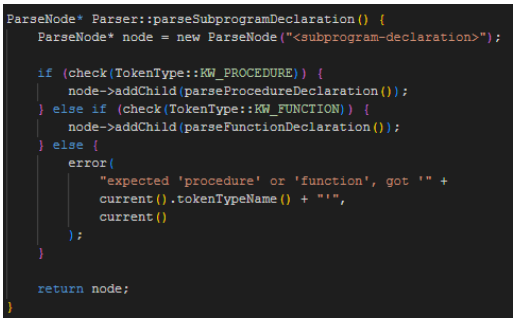
Kelas/Fungsi/Prosedur	**Penjelasan**
ParseNode* Parser::parseExpression() `ParseNode* Parser::parseExpression() { ParseNode* node = new ParseNode("<expression>"); node->addChild(parseSimpleExpression()); if (isRelationalOperator(current().type)) { ParseNode* opNode = new ParseNode("<relational-operator>"); opNode->addChild(makeTerminal(consume())); node->addChild(opNode); node->addChild(parseSimpleExpression()); } return node; }`	Melakukan parsing untuk nonterminal <expression> dalam bentuk simple-expression yang dapat diikuti oleh operator relasional dan simple-expression kedua. Fungsi ini membentuk node <relational-operator> ketika token saat ini merupakan operator perbandingan.
ParseNode* Parser::parseSimpleExpression() `ParseNode* Parser::parseSimpleExpression() { ParseNode* node = new ParseNode("<simple-expression>"); if (check(TokenType::OP_PLUS) || check(TokenType::OP_MINUS)) { node->addChild(makeTerminal(consume())); } node->addChild(parseTerm()); while (isAdditiveOperator(current().type)) { ParseNode* opNode = new ParseNode("<additive-operator>"); opNode->addChild(makeTerminal(consume())); node->addChild(opNode); node->addChild(parseTerm()); } return node; }`	Melakukan parsing untuk ekspresi penjumlahan/logika tingkat menengah. Fungsi ini menangani tanda unary plus/minus di awal, kemudian membaca satu term dan rangkaian additive-operator seperti plus, minus, atau or yang diikuti term berikutnya
ParseNode* Parser::parseTerm()	Melakukan parsing untuk bagian ekspresi dengan prioritas lebih tinggi dari simple-expression. Fungsi ini membaca factor pertama, lalu memproses multiplicative-operator seperti times, rdiv, div, mod, atau and beserta factor berikutnya secara berulang.

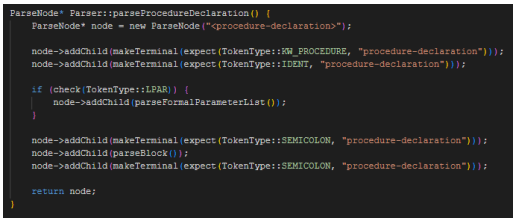
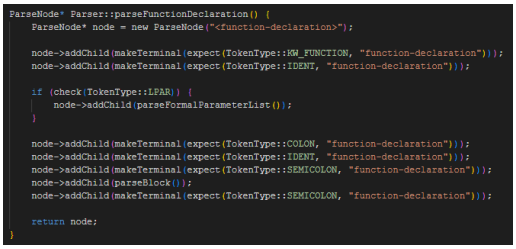
	
ParseNode* Parser::parseFactor() 	Melakukan parsing untuk unit terkecil dalam ekspresi, yaitu variable, literal intcon/realcon/charcon/string, ekspresi dalam tanda kurung, unary not, serta procedure/function-call di dalam ekspresi. Fungsi ini memakai lookahead setelah ident untuk membedakan identifier biasa, component variable, dan pemanggilan fungsi.
ParseNode* Parser::parseParameterList() 	Melakukan parsing untuk daftar parameter aktual pada pemanggilan procedure/function. Parameter pertama diproses sebagai expression, kemudian parameter tambahan dibaca selama terdapat token comma.
bool isRelationalOperator(TokenType type)	Mengecek apakah suatu token termasuk operator relasional, yaitu eql, neq, gtr, geq, lss, atau leq. Helper ini digunakan oleh parseExpression().

	
bool isAdditiveOperator(TokenType type) 	Mengecek apakah suatu token termasuk operator aditif/logika tingkat simple-expression, yaitu plus, minus, atau or. Helper ini digunakan oleh parseSimpleExpression().
bool isMultiplicativeOperator(TokenType type) 	Mengecek apakah suatu token termasuk operator perkalian/pembagian/logika tingkat term, yaitu times, rdiv, div, mod, atau and. Helper ini digunakan oleh parseTerm().

2.4.7. File parser_subprograms.cpp

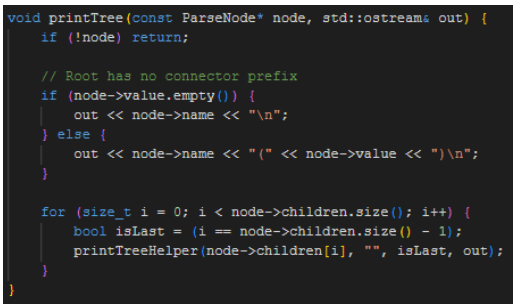
Tabel 2.4.7. Implementasi kelas parser_subprograms

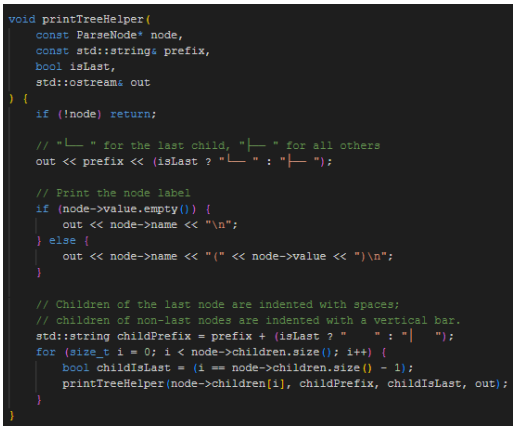
Kelas/Fungsi/Prosedur	**Penjelasan**
ParseNode* Parser::parseSubprogramDeclaration() 	Melakukan dispatch untuk deklarasi subprogram dengan melihat token pertama. Jika token adalah proceduresy maka parser memanggil parseProcedureDeclaration(), sedangkan jika token adalah functionsy maka parser memanggil parseFunctionDeclaration().
ParseNode* Parser::parseProcedureDeclaration()	Melakukan parsing untuk deklarasi procedure dengan pola proceduresy ident, formal-parameter-list

	opsional, semicolon, block, dan semicolon penutup. Fungsi ini memanggil parseFormalParameterList() hanya ketika parameter formal diawali token lparent.
ParseNode* Parser::parseFunctionDeclaration() 	Melakukan parsing untuk deklarasi function dengan pola functionsy ident, formal-parameter-list opsional, colon, ident sebagai tipe kembalian, semicolon, block, dan semicolon penutup. Fungsi ini memastikan tipe kembalian function tersedia sebelum masuk ke block.

2.4.8. File TreePrinter.hpp dan TreePrinter.cpp

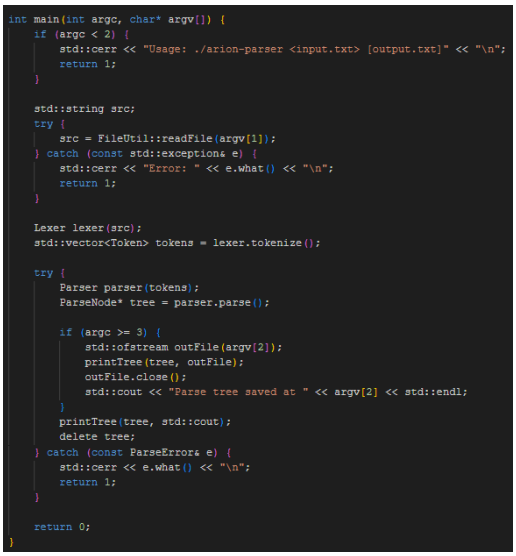
Tabel 2.4.8. Implementasi kelas TreePrinter

Kelas/Fungsi/Prosedur	**Penjelasan**
void printTree(const ParseNode* node, std::ostream& out) 	Fungsi ini menge-*print* prefix ├── pada node root dan └── untuk non-root.
void printTreeHelper(const ParseNode* node, const std::string& prefix, bool isLast, std::ostream& out)	Untuk setiap non-root node, dilakukan *print* rekursif dengan dua kondisi: 1. Apabila node merupakan child terakhir, diberikan empat *space* tanpa "|".

	2. Selain child terakhir, ditambahkan “|”.

2.4.9. File main.cpp

Tabel 2.4.9. Implementasi kelas main

Kelas/Fungsi/Prosedur	**Penjelasan**
int main(int argc, char* argv[]) 	Alur pada program parser dimulai dengan membaca string untuk setiap line. Lalu, setiap string dilakukan tokenizing dengan fungsi `lexer.tokenize()`. Selanjutnya, dilakukan pemanggilan fungsi `parser.parse()` untuk setiap token yang dibaca secara sekuensial. Error pada proses parsing akan ditangkap oleh `catch(const ParseError& e)` yang akan memanggil `std::cerr`.

2.5.Alur Kerja Program

Alur kerja *parser* pada program ini adalah sebagai berikut:

1. Program menerima path file source code dalam bahasa pemrograman Arion sebagai command line argument pertama.
2. Fungsi FIleUtil::readFile() membaca seluruh konten file ke dalam string. Jika file tidak ditemukan atau tidak dapat dibaca, program akan melempar exception untuk menghasilkan pesan *error* dan menghentikan program dengan *exit code* 1.
3. String *source code* diproses oleh Lexer yang melakukan *scanning* karakter per karakter untuk mengidentifikasi pola yang sesuai dengan definisi token bahasa Arion. Lalu lexer menghasilkan std::vector<Token>.
4. Vector token diteruskan ke konstruktor Parser yang menginisialisasi internal state Parser dengan menyimpan *reference* ke token vector dan mengubah posisi *pointer* ke indeks 0 sebagai persiapan proses *parsing*.
5. Proses parsing dimulai melalui pemanggilan Parser::parse(), yang secara internal memanggil parseProgram() sebagai start symbol sesuai grammar, lalu memanggil method parsing lain secara berurutan mengikuti production rule.
6. Setiap method parsing memvalidasi token berdasarkan *production rule* terkait, menggunakan expect() untuk memastikan kesesuaian token, kemudian membentuk node terminal dan menambahkannya sebagai child pada parse tree.
7. Untuk production dengan alternatif, parser menggunakan *lookahead* untuk menentukan rule yang sesuai. Sementara itu, *epsilon production* ditangani dengan kondisi atau *loop* yang berhenti ketika token tidak memenuhi pola awal *rule*.
8. *Parse tree* dibangun secara rekursif dengan membuat ParseNode untuk setiap nonterminal. Lalu, method *parsing* lain dipanggil untuk membentuk *subtree*. Sedangkan terminal node dibuat melalui konversi token dengan penanda isTerminal = true.
9. Validasi grammar dilakukan di setiap langkah menggunakan expect(), jika tidak sesuai, maka ParseError akan dilempar dan ditangani di main.
10. Setelah *parsing* selesai, parser memastikan tidak ada token yang tersisa selain EOF. Jika masih ada token setelah simbol akhir, program akan melempar *error*. Jika tidak, maka program akan mengembalikan *root parse tree*.
11. Setelah seluruh proses selesai, program akan memanggil *destructor* ParseNode untuk menghapus seluruh node secara rekursif dari memori, dan program diakhiri dengan *exit code* 0.

2.6.Justifikasi Implementasi

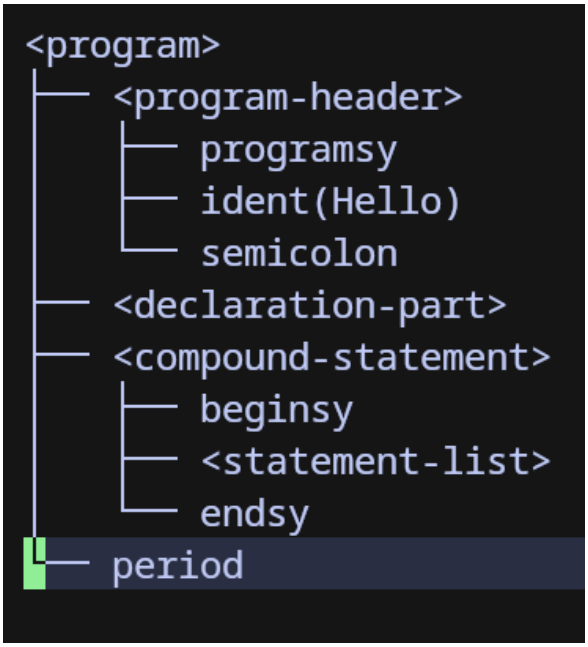
Program diimplementasikan menggunakan bahasa pemrograman C++ karena memberikan kontrol memori yang eksplisit dan performa eksekusi yang tinggi, sehingga pengolahan struktur data hierarkis seperti *parse tree* tidak ada *overhead* tambahan. Pendekatan *parsing* yang digunakan adalah *recursive descent* karena dapat memetakan setiap *production rule* dalam grammar bahasa Arion ke dalam fungsi secara langsung. Representasi *parse tree* menggunakan struktur berbasis *pointer* dengan std::vector untuk menampung *children* secara dinamis. Struktur *parse tree* yang digunakan fleksibel terhadap variasi jumlah node. Selain itu, penanganan *error* dilakukan menggunakan *custom exception* ParseError yang memungkinkan penyampaian pesan kesalahan secara informatif. Sedangkan penggunaan *lookahead* membantu *parser* menemukan *production rule* tanpa harus melakukan *backtracking*.

BAB 3

PENGUJIAN

Tabel 3.1. Pengujian program

No	Input	Output

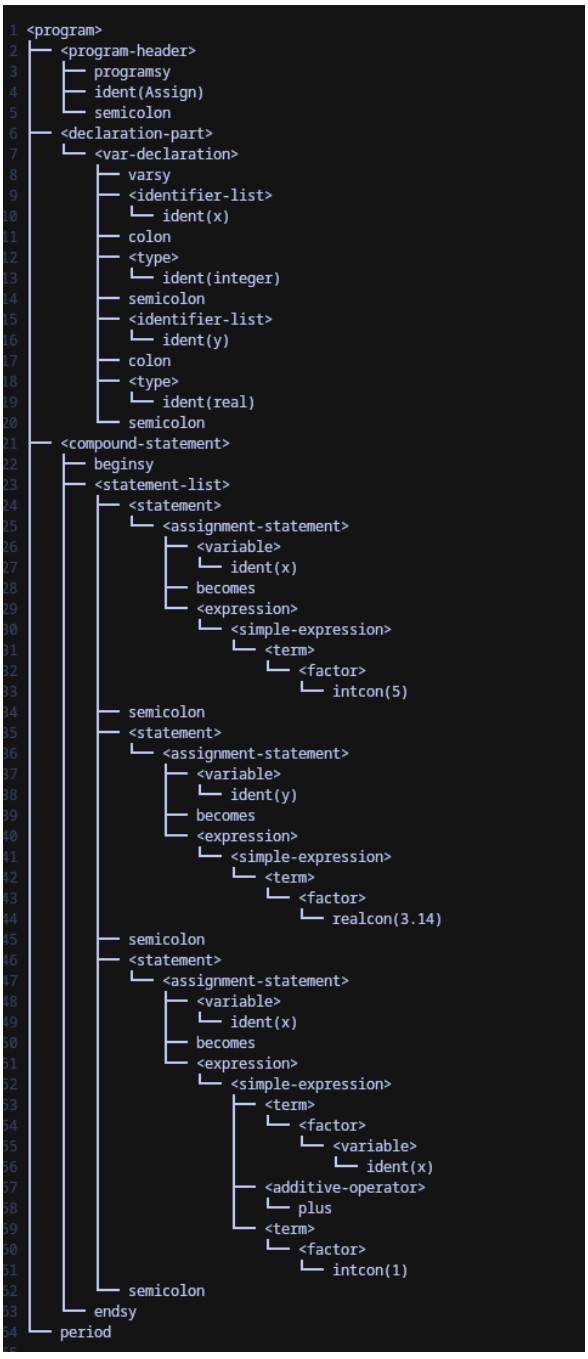
1	Kasus: Masukan valid berupa program yang hanya terdiri dari header, blok deklarasi kosong, dan blok pernyataan kosong.
	`1 program Hello;`
	`2 begin`
	`3 end.`
		
		Parser berhasil memvalidasi bahwa struktur program paling minimal merupakan *syntax* yang sah. Parse tree terbentuk dengan tiga simpul utama, yaitu <program-header>, <declaration-part>, <compound-statement>, dan diakhiri dengan token period(.).

2	Kasus: Masukan valid berupa program yang mendeklarasikan tiga variabel.
	`program Vars;`
	`var`
	`  x: integer;`
	`  a, b: real;`
	`begin`
	`end.`
		<program>
		├── <program-header>
		│ ├── programsy
		│ ├── ident(Vars)
		│ └── semicolon
		├── <declaration-part>
		│ └── <var-declaration>
		│ ├── varsy
		│ ├── <identifier-list>
		│ │ └── ident(x)
		│ ├── colon
		│ ├── <type>
		│ │ └── ident(integer)
		│ ├── semicolon
		│ ├── <identifier-list>
		│ │ ├── ident(a)
		│ │ ├── comma
		│ │ └── ident(b)
		│ ├── colon
		│ ├── <type>
		│ │ └── ident(real)
		│ └── semicolon
		├── <compound-statement>
		│ ├── beginsy
		│ ├── <statement-list>
		│ └── endsy
		└── period
		Parser berhasil mengenali blok var beserta beberapa kelompok deklarasi variabel di dalamnya. Parse tree yang terbentuk menunjukkan <declaration-part> yang berisi sebuah <var-declaration>

3	Kasus: Masukan valid berupa program yang mendeklarasikan tiga konstanta, dengan operator penugasan menggunakan == bukan :=.
	`program ConstTest;`
	`const`
	`  max == 100;`
	`  min == -1;`
	`  msg == 'hello';`
	`begin`
	`end.`
		<program>
		├── <program-header>
		│ ├── programsy
		│ ├── ident(ConstTest)
		│ └── semicolon
		├── <declaration-part>
		│ └── <const-declaration>
		│ ├── constsy
		│ ├── ident(max)
		│ ├── eql
		│ ├── <constant>
		│ │ └── intcon(100)
		│ ├── semicolon
		│ ├── ident(min)
		│ ├── eql
		│ ├── <constant>
		│ │ ├── minus
		│ │ └── intcon(1)
		│ ├── semicolon
		│ ├── ident(msg)
		│ ├── eql
		│ ├── <constant>
		│ │ └── string('hello')
		│ └── semicolon
		├── <compound-statement>
		│ ├── beginsy
		│ ├── <statement-list>
		│ └── endsy
		└── period
		Parser berhasil mengenali blok const beserta tiga jenis nilai konstanta yang berbeda: bilangan bulat positif, bilangan bulat negatif (dengan tanda unary minus yang direpresentasikan sebagai minus(-)), dan literal string. Parse tree yang terbentuk menunjukkan <const-declaration> dengan node <constant> untuk setiap nilai, serta memverifikasi bahwa literal negatif diparsing sebagai dua token (minus dan intcon) di bawah satu simpul <constant>.

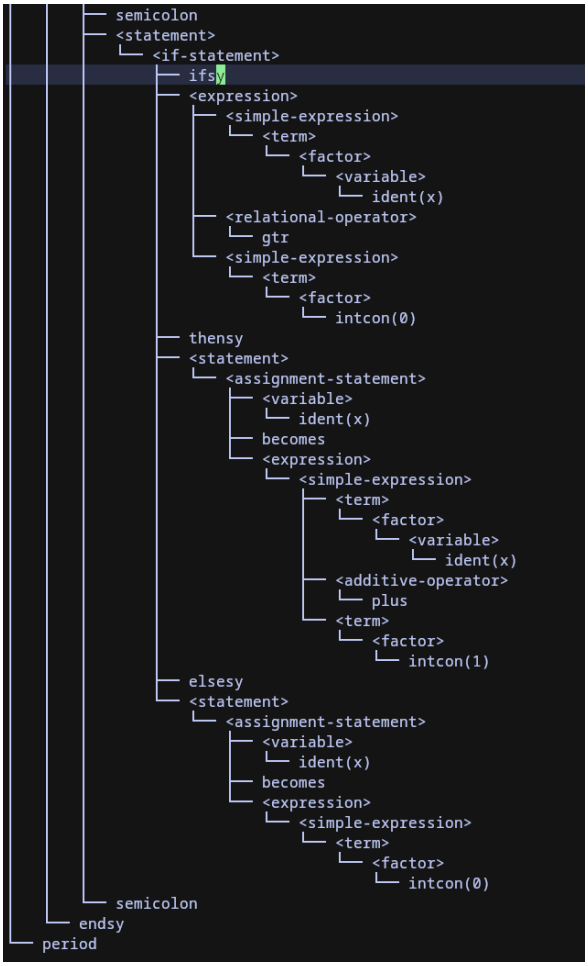
4	Kasus: Masukan valid berupa program yang mendeklarasikan empat jenis 1 program TypesTest; 2 type 3 RangeType == 1 .. 100; 4 Colors == (red, green, blue); 5 IntArray == array [1 .. 10] of integer; 6 Person == record 7 name: string; 8 age: integer 9 end; 10 begin 11 end.	<program> ├─ <program-header> │ ├─ programsy │ ├─ ident(TypesTest) │ └─ semicolon ├─ <declaration-part> │ └─ <type-declaration> │ ├─ typesy │ ├─ ident(RangeType) │ ├─ eql │ ├─ <type> │ │ └─ <range> │ │ ├─ <constant> │ │ │ └─ intcon(1) │ │ ├─ period │ │ ├─ period │ │ └─ <constant> │ │ └─ intcon(100) │ ├─ semicolon │ ├─ ident(Colors) │ ├─ eql │ ├─ <type> │ │ └─ <enumerated> │ │ ├─ lparent │ │ ├─ ident(red) │ │ ├─ comma │ │ ├─ ident(green) │ │ ├─ comma │ │ ├─ ident(blue) │ │ └─ rparent │ ├─ semicolon │ ├─ ident(IntArray) │ ├─ eql │ ├─ <type> │ │ └─ <array-type> │ │ ├─ arraysy │ │ ├─ lbrack │ │ ├─ <range> │ │ │ ├─ <constant> │ │ │ │ └─ intcon(1) │ │ │ ├─ period │ │ │ ├─ period │ │ │ └─ <constant> │ │ │ └─ intcon(10) │ │ ├─ rbrack │ │ ├─ ofsy │ │ └─ <type> │ │ └─ ident(integer) │ ├─ semicolon │ ├─ ident(Person) │ ├─ eql │ ├─ <type> │ │ └─ <record-type> │ │ ├─ recordsy │ │ ├─ <field-list> │ │ │ ├─ <field-part> │ │ │ │ ├─ <identifier-list> │ │ │ │ │ └─ ident(name) │ │ │ │ ├─ colon │ │ │ │ └─ <type> │ │ │ │ └─ ident(string) │ │ │ ├─ semicolon │ │ │ └─ <field-part> │ │ │ ├─ <identifier-list> │ │ │ │ └─ ident(age) │ │ │ ├─ colon │ │ │ └─ <type> │ │ │ └─ ident(integer) │ │ └─ endsy │ └─ semicolon ├─ <compound-statement> │ ├─ beginsy │ ├─ <statement-list> │ └─ endsy └─ period

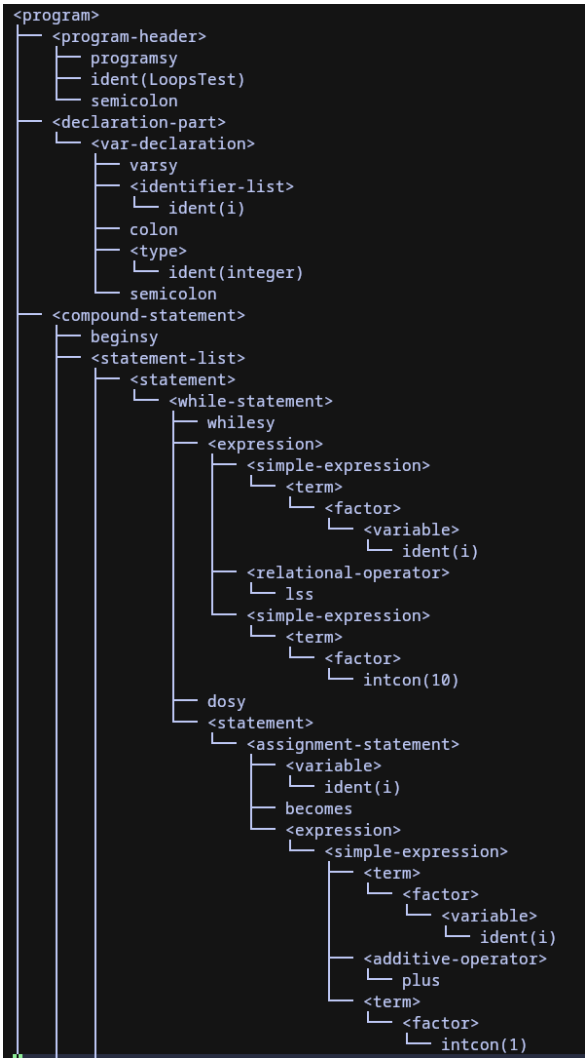
		Parser berhasil mengenali keempat konstruksi tipe yang ada dalam bahasa, yaitu <range>, <enumerated>, <array-type>, dan <record-type>. Parse tree yang terbentuk menunjukkan masing-masing tipe sebagai anak dari node <type> dalam <type-declaration>

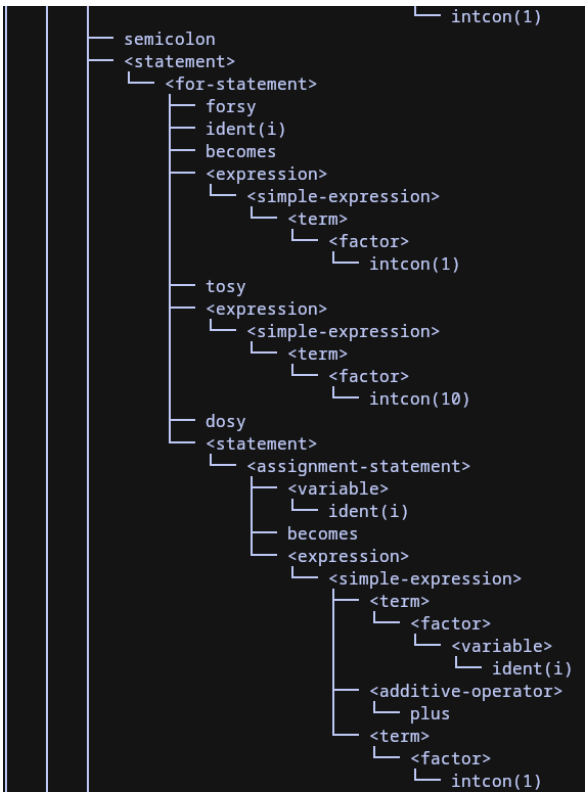
5	Kasus: Masukan valid berupa progran dengan pernyataan penugasan di dalam badan program. Program mendeklarasikan dua variabel, dan melakukan tiga penugasan di badan.
	`program Assign;`
	`var`
	`  x: integer;`
	`  y: real;`
	`begin`
	`  x := 5;`
	`  y := 3.14;`
	`  x := x + 1;`
	`end.`
		

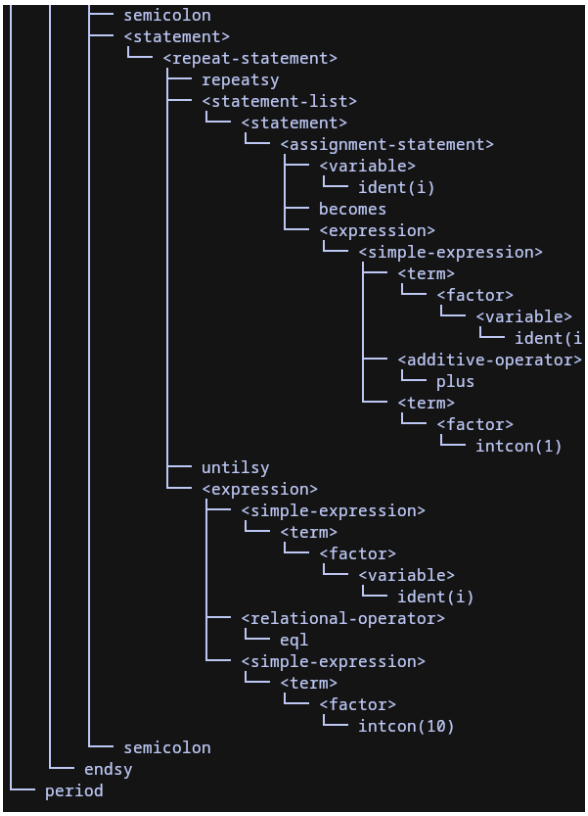
		Parser berhasil mengenali pernyataan penugasan <assignment-statement> dengan beragam jenis ekspresi di sisi kanan. Parse tree yang terbentuk menunjukkan hierarki ekspresi <expression> → <simple-expression> → <term> → <factor> untuk nilai literal, dan penambahan node <additive-operator> dengan token plus(+) untuk ekspresi biner. Selain itu, hasil pengujian ini membuktikan bahwa daftar pernyataan <statement-list> dipisahkan oleh semicolon(;) dengan benar.

6	Kasus: Masukan valid berupa program dengan pernyataan conditional if-then dan if-then-else.
	`program IfElseTest;`
	`var`
	`  x: integer;`
	`begin`
	`  if x == 5 then`
	`  x := 10;`
	`  if x > 0 then`
	`  x := x + 1`
	`  else`
	`  x := 0;`
	`end.`
		

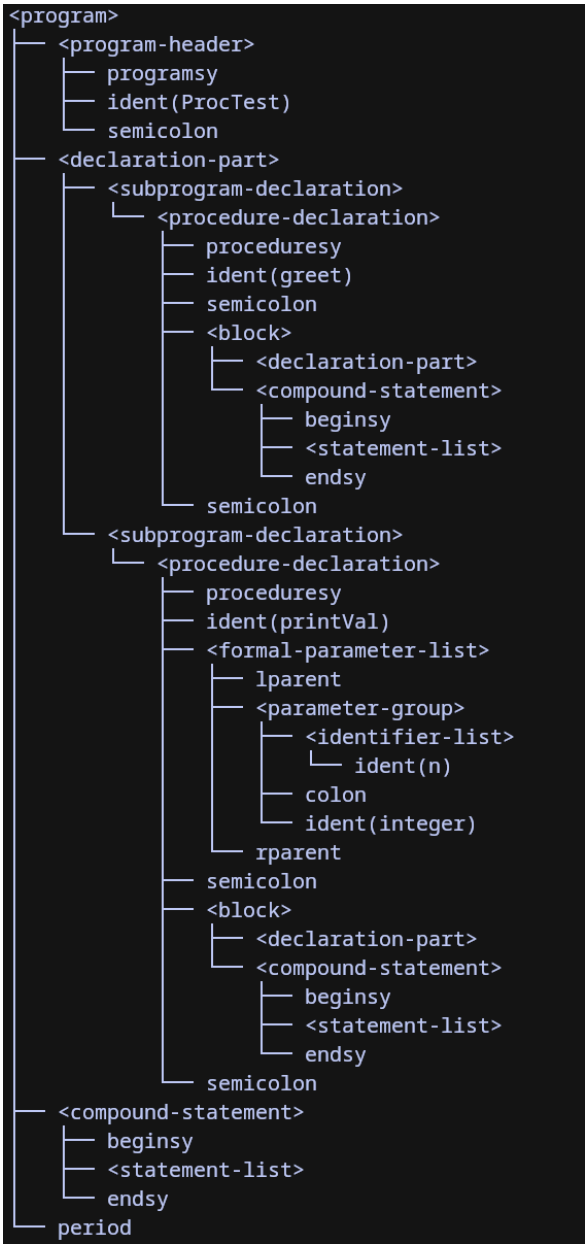
		 Parser berhasil mengenali kedua bentuk pernyataan conditional. Parse tree yang terbentuk menunjukkan <if-statement> dengan node <relational-operator> yang memuat token eql(==) atau gtr(>). Untuk pernyataan if-then-else, terdapat node elsesy(else) setelah pernyataan then. Selain itu, hasil pengujian ini membuktikan bahwa variabel dapat muncul sebagai faktor dalam ekspresi kondisi (<factor> → <variable> → ident(x)).

7	Kasus: Masukan valid berupa program yang menguji keempat jenis pernyataan perulangan (looping). 1 program LoopsTest; 2 var 3 i: integer; 4 begin 5 while i < 10 do 6 i := i + 1; 7 for i := 1 to 10 do 8 i := i + 1; 9 for i := 10 downto 1 do 10 i := i - 1; 11 repeat 12 i := i + 1 13 until i == 10; 14 end.	


intcon(1)
semicolon
<statement>
<for-statement>
forsy
ident(i)
becomes
<expression>
<simple-expression>
<term>
<factor>
intcon(1)
tosy
<expression>
<simple-expression>
<term>
<factor>
intcon(10)
dosy
<statement>
<assignment-statement>
<variable>
ident(i)
becomes
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(i)
<additive-operator>
plus
<term>
<factor>
intcon(1)

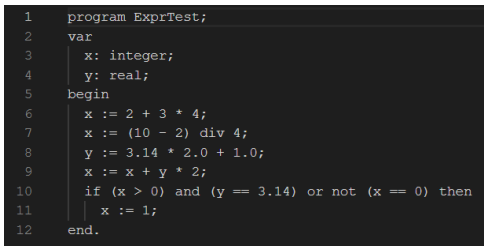
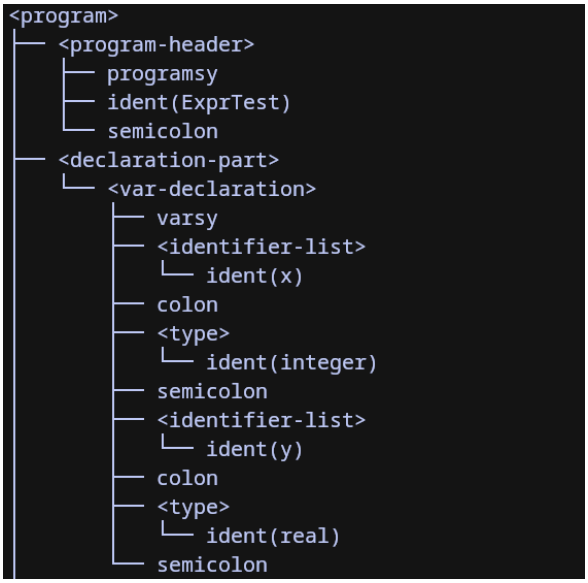

semicolon
<statement>
<repeat-statement>
repeatsy
<statement-list>
<statement>
<assignment-statement>
<variable>
ident(i)
becomes
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(i
<additive-operator>
plus
<term>
<factor>
intcon(1)
untilsy
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(i)
<relational-operator>
eql
<simple-expression>
<term>
<factor>
intcon(10)
semicolon
endsy
period

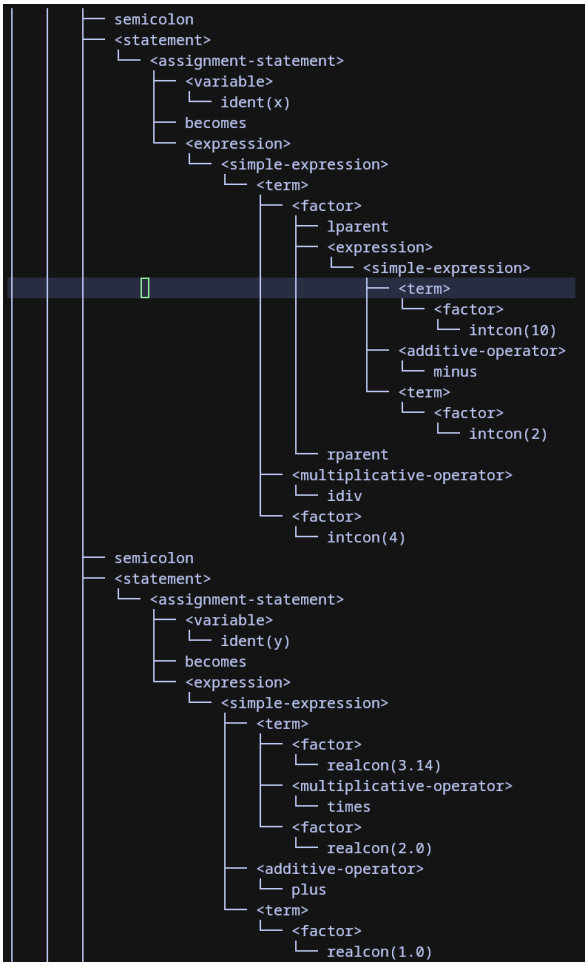
		Parser berhasil memvalidasi keempat jenis perulangan. Parse tree menunjukkan node <while-statement>, <for-statement> (dua kali, masing-masing dengan token tosy(to) dan downtosy(downto)), serta <repeat-statement>. Pernyataan repeat...until bersifat unik karena blok pernyataan di dalamnya menggunakan <statement-list> (bukan <compound-statement>), dan ekspresi kondisi terminalnya direpresentasikan sebagai anak langsung dari node <repeat-statement> .

8	Kasus: Masukan valid berupa program dengan deklarasi dua prosedur pada bagian deklarasi. Prosedur greet dideklarasikan tanpa parameter dan prosedur printVal dideklarasikan dengan sebuah parameter.
	`program ProcTest;`
	`procedure greet;`
	`begin`
	`end;`
	`procedure printVal(n: integer);`
	`begin`
	`end;`
	`begin`
	`end.`
		
		
		Parser berhasil mengenali deklarasi prosedur dalam <declaration-part>, baik untuk prosedur tanpa parameter maupun dengan parameter. Parse tree menunjukkan dua node <subprogram-declaration> yang masing-masing memuat <procedure-declaration>. Prosedur berparameter memiliki <formal-parameter-list> yang berisi <parameter-group> (daftar pengenal,

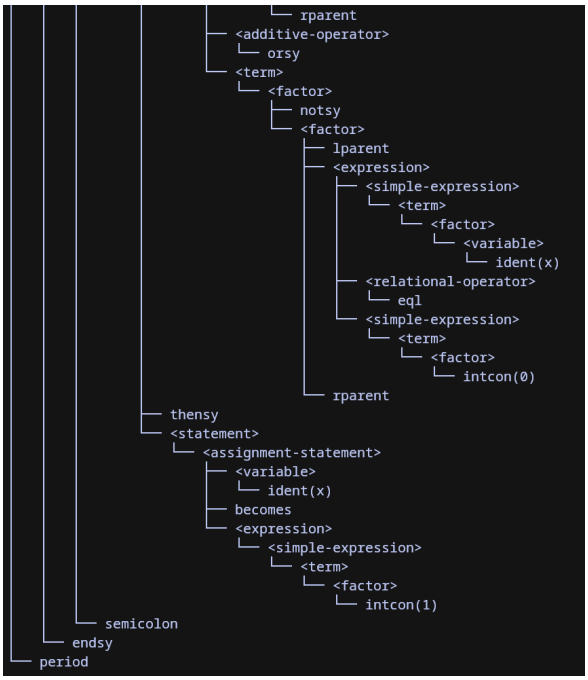
		titik dua, dan tipe). Setiap prosedur diakhiri dengan semicolon(;) setelah bloknya.

9	Kasus: Masukan valid berupa program dengan deklarasi sebuah fungsi yang mengembalikan nilai.
	`program FuncTest;`
	`function add(a: integer; b: integer): integer;`
	`begin`
	`end;`
	`begin`
	`end.`
		<program>
		├── <program-header>
		│ ├── programsy
		│ ├── ident(FuncTest)
		│ └── semicolon
		├── <declaration-part>
		│ └── <subprogram-declaration>
		│ └── <function-declaration>
		│ ├── functionsy
		│ ├── ident(add)
		│ ├── <formal-parameter-list>
		│ │ ├── lparent
		│ │ ├── <parameter-group>
		│ │ │ ├── <identifier-list>
		│ │ │ │ └── ident(a)
		│ │ │ ├── colon
		│ │ │ └── ident(integer)
		│ │ ├── semicolon
		│ │ ├── <parameter-group>
		│ │ │ ├── <identifier-list>
		│ │ │ │ └── ident(b)
		│ │ │ ├── colon
		│ │ │ └── ident(integer)
		│ │ └── rparent
		│ ├── colon
		│ ├── ident(integer)
		│ ├── semicolon
		│ ├── <block>
		│ │ ├── <declaration-part>
		│ │ └── <compound-statement>
		│ │ ├── beginsy
		│ │ ├── <statement-list>
		│ │ └── endsy
		│ └── semicolon
		├── <compound-statement>
		│ ├── beginsy
		│ ├── <statement-list>
		│ └── endsy
		└── period
		Parser berhasil mengenali deklarasi fungsi, yang dibedakan dari prosedur karena adanya tipe yang harus dikembalikan setelah tanda titik dua. Parse tree menunjukkan <function-declaration> dengan <formal-parameter-list> yang memuat dua <parameter-group> yang dipisahkan oleh semicolon(;), serta token colon(:) dan

		ident(integer) sebagai representasi tipe kembalian sebelum semicolon(;) penutup header.
10	Kasus: Masukan valid berupa program yang menguji berbagai bentuk ekspresi aritmetika dan logika.  1 program ExprTest; 2 var 3 x: integer; 4 y: real; 5 begin 6 x := 2 + 3 * 4; 7 x := (10 - 2) div 4; 8 y := 3.14 * 2.0 + 1.0; 9 x := x + y * 2; 10 if (x > 0) and (y == 3.14) or not (x == 0) then 11 x := 1; 12 end.	 <program> ├── <program-header> │ ├── programsy │ ├── ident(ExprTest) │ └── semicolon ├── <declaration-part> │ └── <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ └── ident(x) │ ├── colon │ ├── <type> │ │ └── ident(integer) │ ├── semicolon │ ├── <identifier-list> │ │ └── ident(y) │ ├── colon │ ├── <type> │ │ └── ident(real) │ └── semicolon  ├── <compound-statement> │ ├── beginsy │ ├── <statement-list> │ │ ├── <statement> │ │ │ └── <assignment-statement> │ │ │ ├── <variable> │ │ │ │ └── ident(x) │ │ │ ├── becomes │ │ │ └── <expression> │ │ │ └── <simple-expression> │ │ │ ├── <term> │ │ │ │ └── <factor> │ │ │ │ └── intcon(2) │ │ │ ├── <additive-operator> │ │ │ │ └── plus │ │ │ └── <term> │ │ │ ├── <factor> │ │ │ │ └── intcon(3) │ │ │ ├── <multiplicative-operator> │ │ │ │ └── times │ │ │ └── <factor> │ │ │ └── intcon(4)


semicolon
<statement>
<assignment-statement>
<variable>
ident(x)
becomes
<expression>
<simple-expression>
<term>
<factor>
lparent
<expression>
<simple-expression>
<term>
<factor>
intcon(10)
<additive-operator>
minus
<term>
<factor>
intcon(2)
rparent
<multiplicative-operator>
idiv
<factor>
intcon(4)
semicolon
<statement>
<assignment-statement>
<variable>
ident(y)
becomes
<expression>
<simple-expression>
<term>
<factor>
realcon(3.14)
<multiplicative-operator>
times
<factor>
realcon(2.0)
<additive-operator>
plus
<term>
<factor>
realcon(1.0)

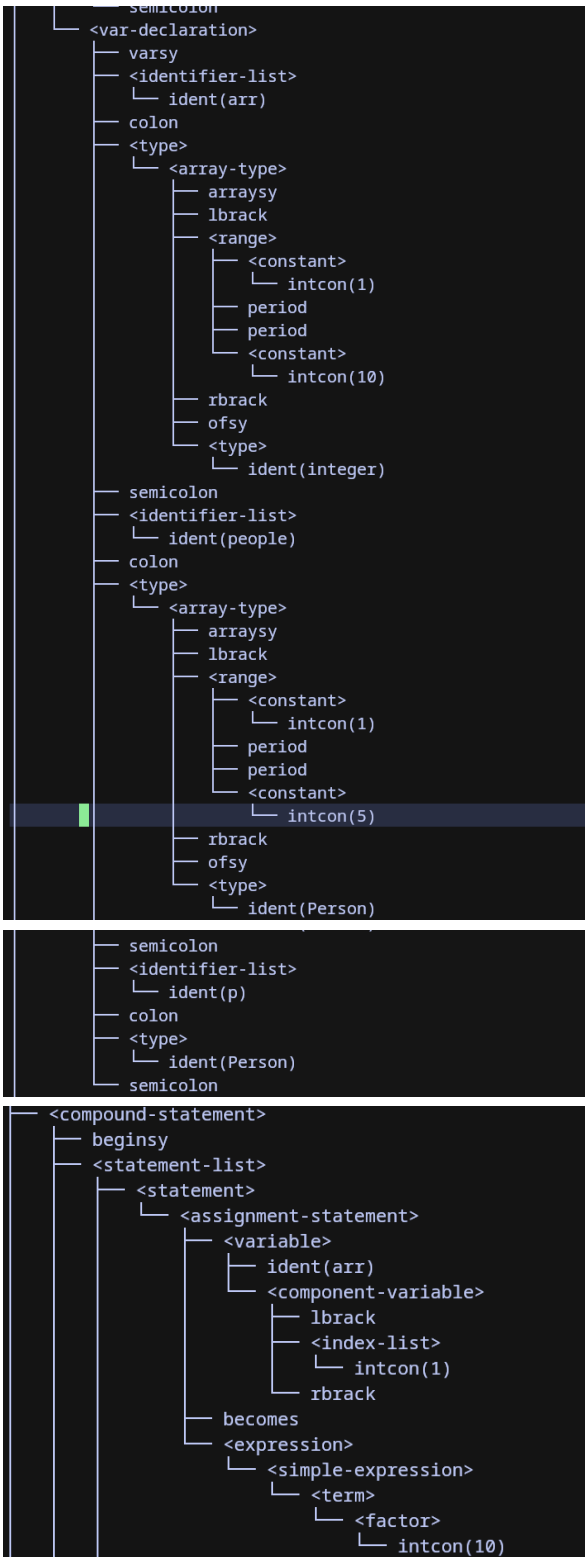

semicolon
<statement>
<assignment-statement>
<variable>
ident(x)
becomes
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(x)
<additive-operator>
plus
<term>
<factor>
<variable>
ident(y)
<multiplicative-operator>
times
<factor>
intcon(2)

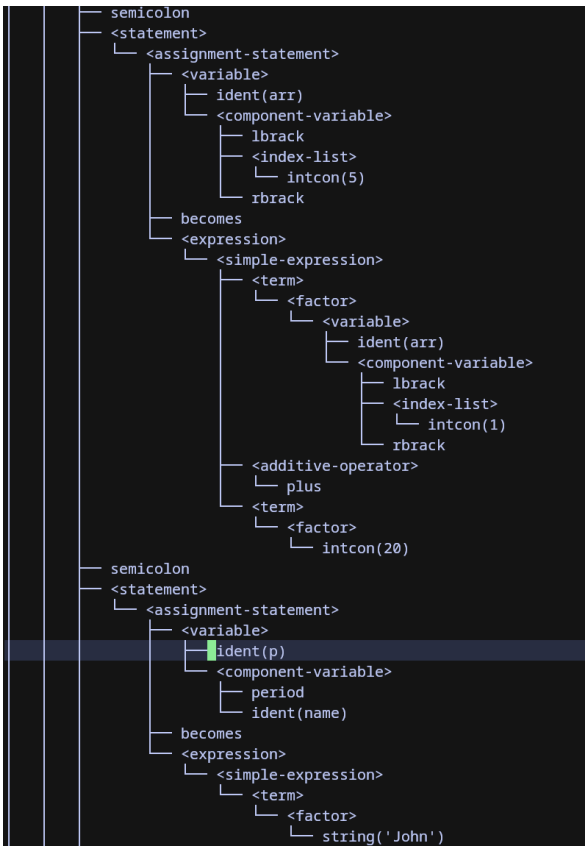


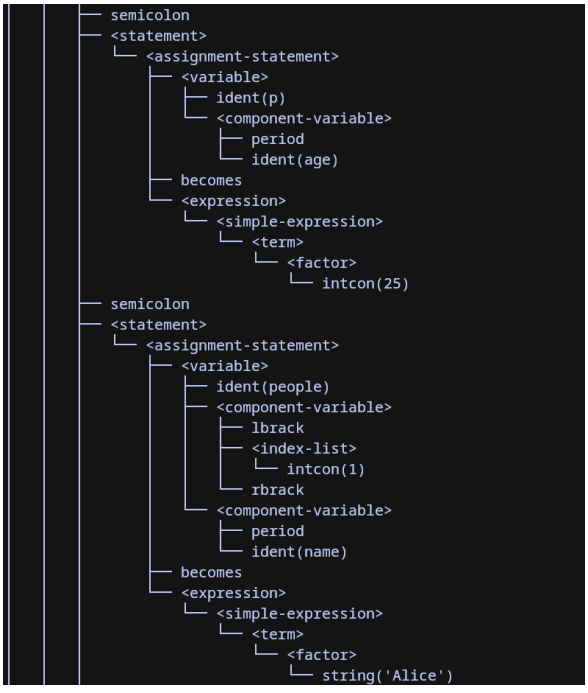
Parser berhasil memvalidasi hierarki prioritas operator yang tercermin dalam struktur parse tree, yaitu operator perkalian (times, div, andsy) berada lebih dalam (sebagai <multiplicative-operator> dalam <term>)

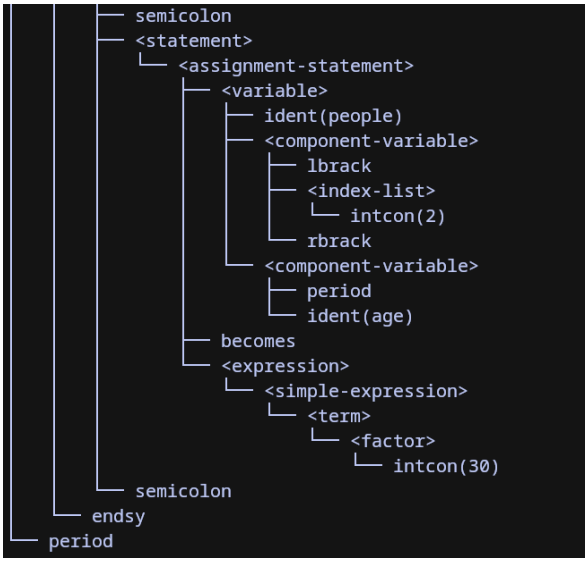
		dibandingkan operator penjumlahan (plus, minus, orsy) yang berada sebagai <additive-operator> dalam <simple-expression>. Operator not diparsing sebagai <factor> unary dengan satu anak <factor>.

11	Kasus: Masukan valid berupa program yang menguji akses dan *indexing* elemen *array* dan *record-field*.
	`program ArrayRecordTest;`
	`type`
	`  Person == record`
	`  name: string;`
	`  age: integer`
	`  end;`
	`var`
	`  arr: array[1..10] of integer;`
	`  people: array[1..5] of Person;`
	`  p: Person;`
	`begin`
	`  arr[1] := 10;`
	`  arr[5] := arr[1] + 20;`
	`  p.name := 'John';`
	`  p.age := 25;`
	`  people[1].name := 'Alice';`
	`  people[2].age := 30;`
	`end.`
		<program>
		├── <program-header>
		│ ├── programsy
		│ ├── ident(ArrayRecordTest)
		│ └── semicolon
		├── <declaration-part>
		│ ├── <type-declaration>
		│ ├── typesy
		│ ├── ident(Person)
		│ ├── eql
		│ ├── <type>
		│ └── <record-type>
		│ ├── recordsy
		│ ├── <field-list>
		│ ├── <field-part>
		│ ├── <identifier-list>
		│ └── ident(name)
		│ ├── colon
		│ └── <type>
		│ └── ident(string)
		│ ├── semicolon
		│ └── <field-part>
		│ ├── <identifier-list>
		│ └── ident(age)
		│ ├── colon
		│ └── <type>
		│ └── ident(integer)
		│ └── endsy
		└── semicolon


semicolon
<var-declaration>
varsy
<identifier-list>
ident(arr)
colon
<type>
<array-type>
arraysy
lbrack
<range>
<constant>
intcon(1)
period
period
<constant>
intcon(10)
rbrack
ofsy
<type>
ident(integer)
semicolon
<identifier-list>
ident(people)
colon
<type>
<array-type>
arraysy
lbrack
<range>
<constant>
intcon(1)
period
period
<constant>
intcon(5)
rbrack
ofsy
<type>
ident(Person)
semicolon
<identifier-list>
ident(p)
colon
<type>
ident(Person)
semicolon
<compound-statement>
beginsy
<statement-list>
<statement>
<assignment-statement>
<variable>
ident(arr)
<component-variable>
lbrack
<index-list>
intcon(1)
rbrack
becomes
<expression>
<simple-expression>
<term>
<factor>
intcon(10)


semicolon
<statement>
<assignment-statement>
<variable>
ident(arr)
<component-variable>
lbrack
<index-list>
intcon(5)
rbrack
becomes
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(arr)
<component-variable>
lbrack
<index-list>
intcon(1)
rbrack
<additive-operator>
plus
<term>
<factor>
intcon(20)
semicolon
<statement>
<assignment-statement>
<variable>
ident(p)
<component-variable>
period
ident(name)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('John')


semicolon
<statement>
<assignment-statement>
<variable>
ident(p)
<component-variable>
period
ident(age)
becomes
<expression>
<simple-expression>
<term>
<factor>
intcon(25)
semicolon
<statement>
<assignment-statement>
<variable>
ident(people)
<component-variable>
lbrack
<index-list>
intcon(1)
rbrack
<component-variable>
period
ident(name)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Alice')

		 Parser berhasil memvalidasi kemampuan mengenali akses komponen variabel <component-variable> yang merupakan anak dari node <variable>. Akses *array* menghasilkan <component-variable> dengan lbrack([), <index-list>, dan rbrack(]). Akses *record field* menghasilkan <component-variable> dengan period(.) dan ident(nama_field). Akses berantai pada *array of record* people[1].name menghasilkan dua node <component-variable> yang bertingkat di bawah satu <variable>

12	Kasus: Masukan valid berupa program yang menguji pernyataan case … of … end. 1 program CaseStatementTest; 2 var 3 day: integer; 4 month: integer; 5 result: string; 6 begin 7 case day of 8 1: result := 'Monday'; 9 2: result := 'Tuesday'; 10 3, 4, 5: result := 'Midweek'; 11 6: result := 'Saturday'; 12 7: result := 'Sunday' 13 end; 14 case month of 15 1: result := 'Jan'; 16 2: result := 'Feb' 17 end; 18 end.	<program> ├── <program-header> │ ├── programsy │ ├── ident(CaseStatementTest) │ └── semicolon ├── <declaration-part> │ └── <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ └── ident(day) │ ├── colon │ ├── <type> │ │ └── ident(integer) │ ├── semicolon │ ├── <identifier-list> │ │ └── ident(month) │ ├── colon │ ├── <type> │ │ └── ident(integer) │ ├── semicolon │ ├── <identifier-list> │ │ └── ident(result) │ ├── colon │ ├── <type> │ │ └── ident(string) │ └── semicolon ├── <compound-statement> │ ├── beginsy │ ├── <statement-list> │ │ ├── <statement> │ │ └── <case-statement> │ │ ├── casesy │ │ ├── <expression> │ │ └── <simple-expression> │ │ └── <term> │ │ └── <factor> │ │ └── <variable> │ │ └── ident(day) │ │ ├── <case-block> │ │ ├── <constant> │ │ └── intcon(1) │ │ ├── colon │ │ ├── <statement> │ │ └── <assignment-statement> │ │ ├── <variable> │ │ └── ident(result) │ │ ├── becomes │ │ └── <expression> │ │ └── <simple-expression> │ │ └── <term> │ │ └── <factor> │ │ └── string('Monday') │ │ ├── semicolon │ │ └── <case-block> │ │ ├── <constant> │ │ └── intcon(2) │ │ ├── colon │ │ ├── <statement> │ │ └── <assignment-statement> │ │ ├── <variable> │ │ └── ident(result) │ │ ├── becomes │ │ └── <expression> │ │ └── <simple-expression> │ │ └── <term> │ │ └── <factor> │ │ └── string('Tuesday')


semicolon
<case-block>
<constant>
intcon(3)
comma
<constant>
intcon(4)
comma
<constant>
intcon(5)
colon
<statement>
<assignment-statement>
<variable>
ident(result)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Midweek')
semicolon
<case-block>
<constant>
intcon(6)
colon
<statement>
<assignment-statement>
<variable>
ident(result)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Saturday')


semicolon
<case-block>
<constant>
intcon(7)
colon
<statement>
<assignment-statement>
<variable>
ident(result)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Sunday


endsy
semicolon
<statement>
<case-statement>
casesy
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident(month)
ofsy
<case-block>
<constant>
intcon(1)
colon
<statement>
<assignment-statement>
<variable>
ident(result)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Jan')
semicolon
<case-block>
<constant>
intcon(2)
colon
<statement>
<assignment-statement>
<variable>
ident(result)
becomes
<expression>
<simple-expression>
<term>
<factor>
string('Feb')
endsy
semicolon
endsy
period

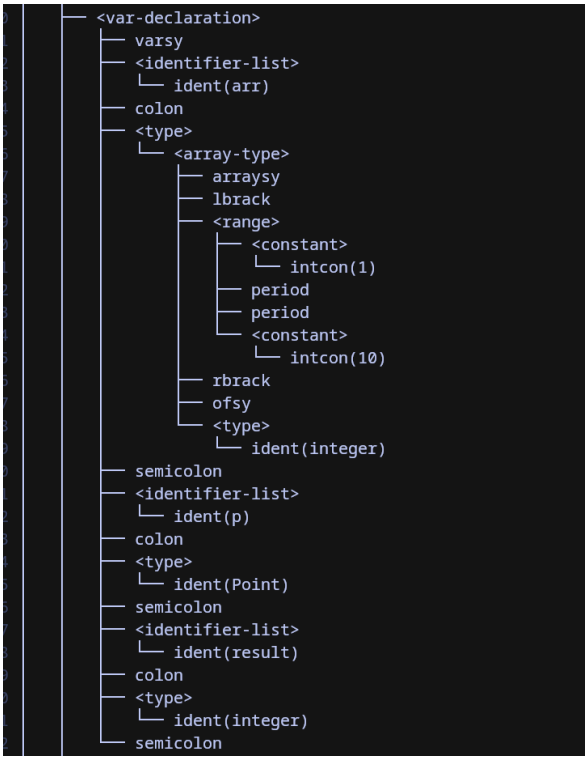
		Parser berhasil mengenali pernyataan case beserta sejumlah cabang di dalamnya. Parse tree menunjukkan node <case-statement> dengan ekspresi selektor, token ofsy(of), dan daftar <case-block> yang direpresentasikan secara rekursif (setiap <case-block> mengandung blok berikutnya sebagai anaknya). Cabang dengan beberapa label (3, 4, 5) direpresentasikan sebagai beberapa node <constant> yang dipisahkan oleh comma(,) sebelum colon(:) dalam satu <case-block>. Pernyataan case ditutup dengan endsy(end).

13	Kasus: Masukan valid berupa program yang menguji pemanggilan prosedur dari dalam badan program maupun dari dalam prosedur lain.
	`program ProcedureCallTest;`
	`procedure greet;`
	`begin`
	`end;`
	
	`procedure printNum(n: integer);`
	`begin`
	`end;`
	
	`procedure compute(a: integer; b: integer);`
	`var`
	`  result: integer;`
	`begin`
	`  result := a + b;`
	`  printNum(result);`
	`end;`
	
	`begin`
	`  greet();`
	`  printNum(5);`
	`  compute(10, 20);`
	`end.`
		


└── <subprogram-declaration>
└── <procedure-declaration>
├── proceduresy
├── ident(compute)
├── <formal-parameter-list>
├── lparent
├── <parameter-group>
├── <identifier-list>
└── ident(a)
├── colon
└── ident(integer)
├── semicolon
├── <parameter-group>
├── <identifier-list>
└── ident(b)
├── colon
└── ident(integer)
└── rparent
├── semicolon
├── <block>
├── <declaration-part>
└── <var-declaration>
├── varsy
├── <identifier-list>
└── ident(result)
├── colon
├── <type>
└── ident(integer)
└── semicolon
└── <compound-statement>
├── beginsy
├── <statement-list>
├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(result)
├── becomes
└── <expression>
└── <simple-expression>
├── <term>
└── <factor>
└── <variable>
└── ident(a)
├── <additive-operator>
└── plus
└── <term>
└── <factor>
└── <variable>
└── ident(b)
├── semicolon
├── <statement>
└── <procedure/function-call>
├── ident(printNum)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
└── ident(result)
└── rparent
└── semicolon
└── endsy
└── semicolon
├── <compound-statement>
├── beginsy
├── <statement-list>
├── <statement>
└── <procedure/function-call>
├── ident(greet)
├── lparent
└── rparent
├── semicolon
├── <statement>
└── <procedure/function-call>
├── ident(printNum)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── intcon(5)
└── rparent
├── semicolon
├── <statement>
└── <procedure/function-call>
├── ident(compute)
├── lparent
├── <parameter-list>
├── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── intcon(10)
├── comma
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── intcon(20)
└── rparent
└── semicolon
└── endsy
└── period

		Parser berhasil mengenali <procedure/function-call> sebagai bentuk pernyataan (<statement>). Pemanggilan tanpa argumen hanya memiliki ident(greet) sebagai satu-satunya anak. Pemanggilan dengan argumen memiliki lparent(()、<parameter-list> (berisi satu atau lebih <expression> yang dipisahkan comma(,)), dan rparent()). Selain itu, hasil pengujian ini juga membuktikan bahwa pemanggilan prosedur dapat terjadi di dalam blok prosedur lain, dan bahwa deklarasi variabel lokal dalam prosedur dapat diparsing dengan benar.

14	Kasus: Masukan valid berupa program yang menguji kombinasi ekspresi kompleks yang melibatkan pemanggilan fungsi, akses *array*, dan akses *record field* secara bersamaan dalam satu ekspresi.
	`program ComplexExprTest;`
	`type`
	`  Point == record`
	`  x: integer;`
	`  y: integer`
	`  end;`
	`var`
	`  arr: array[1..10] of integer;`
	`  p: Point;`
	`  result: integer;`
	`function square(n: integer): integer;`
	`begin`
	`end;`
	`function distance(p1: Point; p2: Point): integer;`
	`begin`
	`end;`
	`begin`
	`  result := square(5) + arr[1];`
	`  result := arr[2] * square(p.x);`
	`  arr[3] := square(arr[1]) + square(arr[2]);`
	`  p.x := square(10);`
	`  result := p.x + p.y * arr[1];`
	`end.`
		<program>
		├── <program-header>
		│ ├── programsy
		│ ├── ident(ComplexExprTest)
		│ └── semicolon
		├── <declaration-part>
		│ ├── <type-declaration>
		│ │ ├── typesy
		│ │ ├── ident(Point)
		│ │ ├── eql
		│ │ ├── <type>
		│ │ │ └── <record-type>
		│ │ │ ├── recordsy
		│ │ │ ├── <field-list>
		│ │ │ │ ├── <field-part>
		│ │ │ │ │ ├── <identifier-list>
		│ │ │ │ │ │ └── ident(x)
		│ │ │ │ │ ├── colon
		│ │ │ │ │ └── <type>
		│ │ │ │ │ └── ident(integer)
		│ │ │ │ ├── semicolon
		│ │ │ │ └── <field-part>
		│ │ │ │ ├── <identifier-list>
		│ │ │ │ │ └── ident(y)
		│ │ │ │ ├── colon
		│ │ │ │ └── <type>
		│ │ │ │ └── ident(integer)
		│ │ │ └── endsy
		│ │ └── semicolon


├── <var-declaration>
│ ├── varsy
│ ├── <identifier-list>
│ │ └── ident(arr)
│ ├── colon
│ ├── <type>
│ │ └── <array-type>
│ │ ├── arraysy
│ │ ├── lbrack
│ │ ├── <range>
│ │ │ ├── <constant>
│ │ │ │ └── intcon(1)
│ │ │ ├── period
│ │ │ ├── period
│ │ │ └── <constant>
│ │ │ └── intcon(10)
│ │ ├── rbrack
│ │ ├── ofsy
│ │ └── <type>
│ │ └── ident(integer)
│ ├── semicolon
│ ├── <identifier-list>
│ │ └── ident(p)
│ ├── colon
│ ├── <type>
│ │ └── ident(Point)
│ ├── semicolon
│ ├── <identifier-list>
│ │ └── ident(result)
│ ├── colon
│ ├── <type>
│ │ └── ident(integer)
│ └── semicolon


├── <subprogram-declaration>
│ └── <function-declaration>
│ ├── functionsy
│ ├── ident(square)
│ ├── <formal-parameter-list>
│ │ ├── lparent
│ │ ├── <parameter-group>
│ │ │ ├── <identifier-list>
│ │ │ │ └── ident(n)
│ │ │ ├── colon
│ │ │ └── ident(integer)
│ │ └── rparent
│ ├── colon
│ ├── ident(integer)
│ ├── semicolon
│ ├── <block>
│ │ ├── <declaration-part>
│ │ └── <compound-statement>
│ │ ├── beginsy
│ │ ├── <statement-list>
│ │ └── endsy
│ └── semicolon
└── <subprogram-declaration>


└── <subprogram-declaration>
└── <function-declaration>
├── functionsy
├── ident(distance)
├── <formal-parameter-list>
├── lparent
├── <parameter-group>
├── <identifier-list>
└── ident(p1)
├── colon
└── ident(Point)
├── semicolon
├── <parameter-group>
├── <identifier-list>
└── ident(p2)
├── colon
└── ident(Point)
└── rparent
├── colon
├── ident(integer)
├── semicolon
├── <block>
├── <declaration-part>
└── <compound-statement>
├── beginsy
├── <statement-list>
└── endsy
└── semicolon


├── <compound-statement>
├── beginsy
├── <statement-list>
├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(result)
├── becomes
└── <expression>
└── <simple-expression>
├── <term>
└── <factor>
└── <procedure/function-call>
├── ident(square)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── intcon(5)
└── rparent
├── <additive-operator>
└── plus
└── <term>
└── <factor>
└── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(1)
└── rbrack


├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(result)
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
├── <factor>
└── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(2)
└── rbrack
├── <multiplicative-operator>
└── times
└── <factor>
└── <procedure/function-call>
├── ident(square)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
├── ident(p)
└── <component-variable>
├── period
└── ident(x)
└── rparent
├── semicolon
├── <statement>
└── <assignment-statement>
├── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(3)
└── rbrack
├── becomes
└── <expression>
└── <simple-expression>
├── <term>
└── <factor>
└── <procedure/function-call>
├── ident(square)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(1)
└── rbrack
└── rparent
├── <additive-operator>
└── plus
└── <term>
└── <factor>
└── <procedure/function-call>
├── ident(square)
├── lparent
├── <parameter-list>
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(2)
└── rbrack
└── rparent
├── semicolon
├── <statement>
└── <assignment-statement>
├── <variable>
├── ident(p)
└── <component-variable>
├── period
└── ident(x)
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>

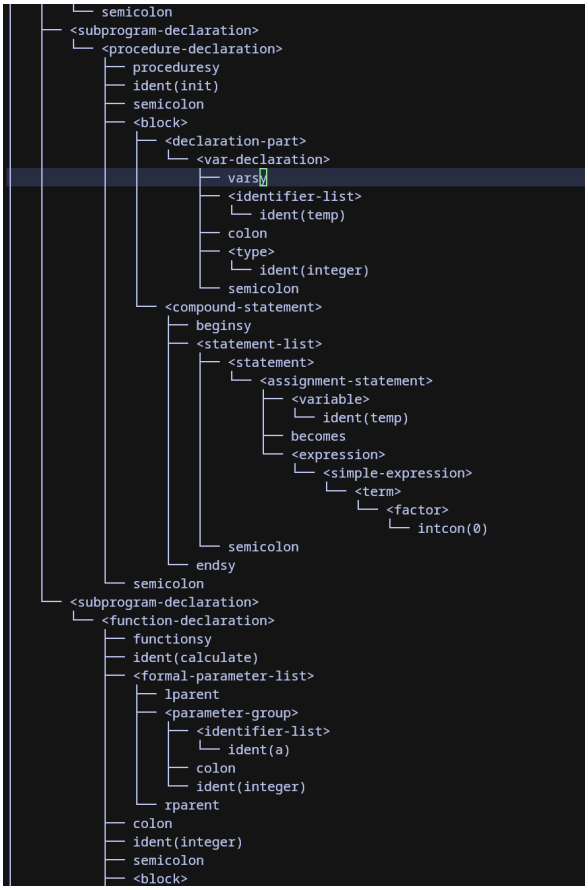
		 Parser berhasil mengenali pemanggilan fungsi <procedure/function-call> sebagai <factor> di dalam ekspresi, yang memungkinkan fungsi digunakan langsung dalam operasi aritmetika tanpa penugasan terlebih dahulu. Parse tree menunjukkan <procedure/function-call> muncul sebagai anak dari <factor> dalam <term>, setara dengan variabel atau literal. Argumen fungsi dapat berupa ekspresi kompleks, seperti arr[1] dan p.x, yang direpresentasikan secara rekursif.

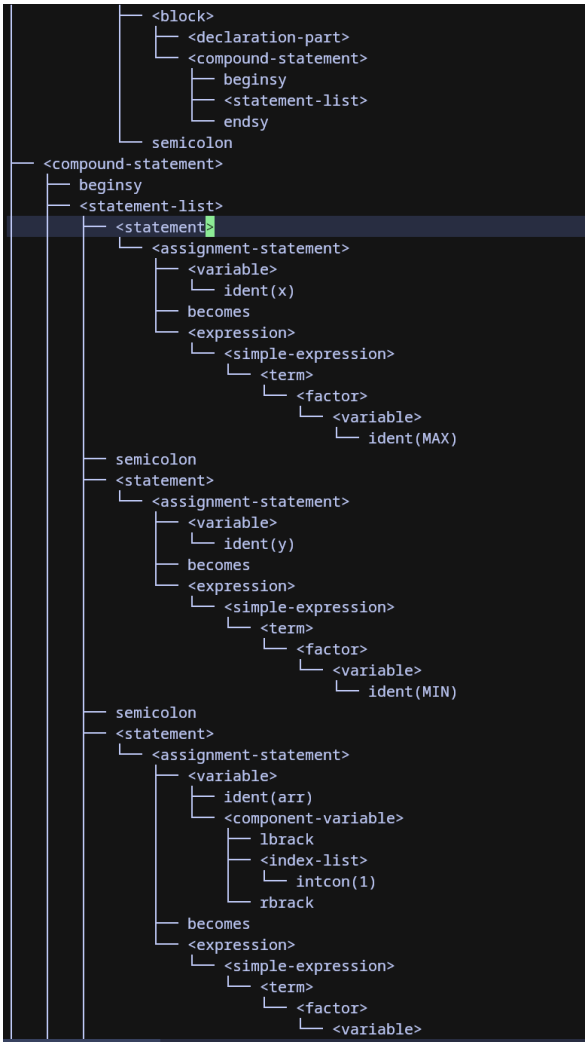
15	Kasus: Masukan valid berupa program yang menggabungkan seluruh jenis deklarasi sekaligus dalam satu program.	

```
program AllDeclTest;
const
  MAX == 100;
  MIN == -10;
  PI == 3.14;
type
  Range == 1..10;
  Colors == (red, green, blue);
  IntArr == array[1..10] of integer;
  Person == record
    name: string;
    age: integer
  end;
var
  x, y: integer;
  c: Colors;
  arr: IntArr;
  p: Person;
procedure init;
var
  temp: integer;
begin
  temp := 0;
end;
function calculate(a: integer): integer;
begin
end;
begin
  x := MAX;
  y := MIN;
  arr[1] := x;
  p.age := 25;
  init();
  y := calculate(x);
end.

```


semicolon
ident(Person)
eql
<type>
<record-type>
recordsy
<field-list>
<field-part>
<identifier-list>
ident(name)
colon
<type>
ident(string)
semicolon
<field-part>
<identifier-list>
ident(age)
colon
<type>
ident(integer)
endsy
semicolon
<var-declaration>
varsy
<identifier-list>
ident(x)
comma
ident(y)
colon
<type>
ident(integer)
semicolon
<identifier-list>
ident(c)
colon
<type>
ident(Colors)
semicolon
<identifier-list>
ident(arr)
colon
<type>
ident(IntArr)
semicolon
<identifier-list>
ident(p)
colon
<type>
ident(Person)


└── semicolon
├── <subprogram-declaration>
└── <procedure-declaration>
├── proceduresy
├── ident(init)
├── semicolon
├── <block>
├── <declaration-part>
└── <var-declaration>
├── varsy
├── <identifier-list>
└── ident(temp)
├── colon
├── <type>
└── ident(integer)
└── semicolon
└── <compound-statement>
├── beginsy
├── <statement-list>
├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(temp)
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── intcon(0)
└── semicolon
└── endsy
└── semicolon
└── <subprogram-declaration>
└── <function-declaration>
├── functionsy
├── ident(calculate)
├── <formal-parameter-list>
├── lparent
├── <parameter-group>
├── <identifier-list>
└── ident(a)
├── colon
└── ident(integer)
└── rparent
├── colon
├── ident(integer)
├── semicolon
├── <block>


├── <block>
│ ├── <declaration-part>
│ └── <compound-statement>
│ ├── beginsy
│ ├── <statement-list>
│ └── endsy
└── semicolon
├── <compound-statement>
├── beginsy
├── <statement-list>
├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(x)
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
└── ident(MAX)
├── semicolon
├── <statement>
└── <assignment-statement>
├── <variable>
└── ident(y)
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>
└── ident(MIN)
├── semicolon
├── <statement>
└── <assignment-statement>
├── <variable>
├── ident(arr)
└── <component-variable>
├── lbrack
├── <index-list>
└── intcon(1)
└── rbrack
├── becomes
└── <expression>
└── <simple-expression>
└── <term>
└── <factor>
└── <variable>

		 Parser berhasil menangani keseluruhan struktur program lengkap secara sekaligus, termasuk urutan bagian deklarasi (<const-declaration>, <type-declaration>, <var-declaration>, dua <subprogram-declaration>) dalam satu <declaration-part>.

16	Kasus: Masukan valid berupa program yang memiliki comment.	  Parser membangun parse tree seperti sesuai dengan susunan program tanpa memasukkan *comment*. Ini dibutuhkan untuk suatu kasus *comment* yang memiliki token yang merupakan token valid pada program.

17	Kasus: Masukan tidak valid berupa if yang berada pada expression dan then yang dilanjutkan dengan semicolon
	
	`program WrongKeyword;`
	`var`
	`  a: integer;`
	`begin`
	`  a := if 5 then;`
	`  a := 3 + ;`
	`end.`
		`~/Documents/Studies/tbfo/JBC-Tubes-IF2224-2026 main 1m 17s`
		`❯ ./arion-parser test/milestone-2/test-18-err-wrong-token.txt output.txt`
		`Syntax error at line 5: unexpected token 'ifsy' in factor - expected expression operand (got 'if')`
		`Syntax error at line 5: in compound-statement: expected endsy, got ifsy (got 'if')`
		`Syntax error at line 5: in program: expected period, got intcon (got '5')`
		`Syntax error at line 5: unexpected token 'thensy' after end of program (got 'then')`
		`Parse tree saved at output.txt`
		`<program>`
		`├── <program-header>`
		`│  ├── programsy`
		`│  ├── ident(WrongKeyword)`
		`│  └── semicolon`
		`├── <declaration-part>`
		`│  └── <var-declaration>`
		`│  ├── varsy`
		`│  ├── <identifier-list>`
		`│  │  └── ident(a)`
		`│  ├── colon`
		`│  ├── <type>`
		`│  │  └── ident(integer)`
		`│  └── semicolon`
		`├── <compound-statement>`
		`│  ├── beginsy`
		`│  ├── <statement-list>`
		`│  │  └── <statement>`
		`│  │  └── <assignment-statement>`
		`│  │  ├── <variable>`
		`│  │  │  └── ident(a)`
		`│  │  ├── becomes`
		`│  │  └── <expression>`
		`│  │  └── <simple-expression>`
		`│  │  └── <term>`
		`│  └── ifsy`
		`└── intcon(5)`

18	Kasus: masukan tidak valid berupa for yang dilanjutkan dengan upto.
	
	`1 program BadFor;`
	`2 var`
	`3   i: integer;`
	`4 begin`
	`5   for i := 1 upto 10 do`
	`6     i := i + 1;`
	`7 end.`
		`❯ ./arion-parser test/milestone-2/test-21-err-bad-for.txt output.txt`
		`Syntax error at line 5: expected 'to' or 'downto' in for-statement (got 'upto')`
		`Syntax error at line 5: in for-statement: expected dosy, got intcon (got '10')`
		`Syntax error at line 5: in compound-statement: expected endsy, got dosy (got 'do')`
		`Syntax error at line 6: in program: expected period, got ident (got 'i')`
		`Syntax error at line 6: unexpected token 'becomes' after end of program (got ':=')`
		`Parse tree saved at output.txt`
		`<program>`
		`├── <program-header>`
		`│  ├── programsy`
		`│  ├── ident(BadFor)`
		`│  └── semicolon`
		`├── <declaration-part>`
		`│  └── <var-declaration>`
		`│  ├── varsy`
		`│  ├── <identifier-list>`
		`│  │  └── ident(i)`
		`│  ├── colon`
		`│  ├── <type>`
		`│  │  └── ident(integer)`
		`│  └── semicolon`
		`├── <compound-statement>`
		`│  ├── beginsy`
		`│  ├── <statement-list>`
		`│  │  └── <statement>`
		`│  │  └── <for-statement>`
		`│  │  ├── forsy`
		`│  │  ├── ident(i)`
		`│  │  ├── becomes`
		`│  │  ├── <expression>`
		`│  │  │  └── <simple-expression>`
		`│  │  │  └── <term>`
		`│  │  │  └── <factor>`
		`│  │  │  └── intcon(1)`
		`│  │  ├── <expression>`
		`│  │  │  └── <simple-expression>`
		`│  │  │  └── <term>`
		`│  │  │  └── <factor>`
		`│  │  │  └── <variable>`
		`│  │  │  └── ident(upto)`
		`│  │  └── intcon(10)`
		`│  └── dosy`
		`└── ident(i)`

BAB 4
KESIMPULAN DAN SARAN

4.1.Kesimpulan

Parser atau *syntax analyzer* bahasa Arion berbasis C++ yang dikembangkan menggunakan pendekatan *recursive descent* telah berhasil diimplementasikan dan berfungsi dengan baik dalam memvalidasi urutan token yang dihasilkan oleh lexical analyzer sesuai dengan *context-free grammar* yang telah didefinisikan. Setiap *production rule* dalam *grammar* dipetakan menjadi method *parsing* terpisah. Arsitektur program yang membagi method *parsing* ke dalam beberapa kategori fungsional, seperti *top level, declarations, statements, expressions,* dan *subprogram,* memungkinkan pengembangan yang modular, terstruktur, dan mudah dipelihara.

Program *parser* dapat membentuk *parse tree* yang merepresentasikan struktur hierarkis program sumber, dengan setiap node menyimpan informasi simbol terminal dan nonterminal beserta *children*-nya. Mekanisme *lookahead* memungkinkan *parser* menangani grammar dengan alternatif *production* dan *epsilon production* tanpa memerlukan *backtracking*. Output *parse tree* dicetak dengan format *tree connector* untuk memudahkan visualisasi struktur program dan verifikasi hasil *parsing*. Selain itu, program melakukan *error handling* menggunakan *exception* ParseError.

4.2.Saran

Spesifikasi *grammar* dan perilaku *parser* perlu didokumentasikan secara konsisten agar implementasi tidak ambigu dan mudah dipelihara pada pengembangan selanjutnya. Penambahan *unit testing* pada tiap metode *parsing* akan meningkatkan keandalan dan memudahkan *regression testing*. Selain itu, mekanisme *error recovery* dapat diterapkan agar *parser* mampu melaporkan beberapa *error* sekaligus tanpa berhenti di kesalahan pertama. Optimisasi memori dengan *smart pointer* juga dapat meningkatkan keamanan program.

BAB 5

LAMPIRAN

5.1.Grammar yang Digunakan

Tabel 5.2. Definisi formal grammar program syntax analyzer bahasa pemrograman Arion

Komponen	**Anggota**
Variables	<program> <program-header> <declaration-part> <block> <const-declaration> <constant> <type-declaration> <type> <array-type> <range> <enumerated> <record-type> <field-list> <field-part> <var-declaration> <identifier-list> <subprogram-declaration> <procedure-declaration> <function-declaration> <formal-parameter-list> <parameter-group> <compound-statement> <statement-list> <statement> <variable> <component-variable> <index-list> <assignment-statement> <if-statement> <case-statement> <case-block> <while-statement> <repeat-statement> <for-statement> <procedure/function-call> <parameter-list> <expression> <simple-expression> <term>

	<factor> <relational-operator> <additive-operator> <multiplicative-operator>
Terminal	KEYWORD(program), KEYWORD(const), KEYWORD(type), KEYWORD(var), KEYWORD(array), KEYWORD(of), KEYWORD(record), KEYWORD(end), KEYWORD(procedure), KEYWORD(function), KEYWORD(begin), KEYWORD(if), KEYWORD(then), KEYWORD(else), KEYWORD(case), KEYWORD(while), KEYWORD(do), KEYWORD(repeat), KEYWORD(until), KEYWORD(for), KEYWORD(to), KEYWORD(downto), IDENTIFIER, INTEGER_LITERAL, REAL_LITERAL, CHAR_LITERAL, STRING_LITERAL, SEMICOLON(;), COLON(:), COMMA(,), PERIOD(.), LPARENTHESIS((), RPARENTHESIS()), LBRACKET([), RBRACKET(]), RELATIONAL_OPERATOR(==), RELATIONAL_OPERATOR(<>), RELATIONAL_OPERATOR(<), RELATIONAL_OPERATOR(<=), RELATIONAL_OPERATOR(>), RELATIONAL_OPERATOR(>=), ASSIGN_OPERATOR(:=), ARITHMETIC_OPERATOR(+), ARITHMETIC_OPERATOR(-), ARITHMETIC_OPERATOR(*),

	ARITHMETIC_OPERATOR(/), ARITHMETIC_OPERATOR(div), ARITHMETIC_OPERATOR(mod), LOGICAL_OPERATOR(and), LOGICAL_OPERATOR(or), LOGICAL_OPERATOR(not)
Productions	<program> ::= <program-header> <declaration-part> <compound-statement> PERIOD <program-header> ::= KEYWORD(program) IDENTIFIER SEMICOLON <declaration-part> ::= { <const-declaration> } { <type-declaration> } { <var-declaration> } { <subprogram-declaration> } <block> ::= <declaration-part> <compound-statement> <const-declaration> ::= KEYWORD(const) (IDENTIFIER RELATIONAL_OPERATOR(==) <constant> SEMICOLON)+ <constant> ::= CHAR_LITERAL | STRING_LITERAL | [ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-)] (IDENTIFIER | INTEGER_LITERAL | REAL_LITERAL) <type-declaration> ::= KEYWORD(type) (IDENTIFIER RELATIONAL_OPERATOR(==) <type> SEMICOLON)+ <type> ::= <array-type> | <enumerated> | <record-type> | IDENTIFIER | <range> <array-type> ::= KEYWORD(array) LBRACKET (<range> | IDENTIFIER) RBRACKET KEYWORD(of) <type> <range> ::= <expression> PERIOD PERIOD <expression> <enumerated> ::= LPARENTHESIS IDENTIFIER { COMMA IDENTIFIER } RPARENTHESIS <record-type> ::= KEYWORD(record) <field-list> KEYWORD(end) <field-list> ::= <field-part> { SEMICOLON <field-part> } [SEMICOLON]

<field-part> ::= <identifier-list> COLON <type>

<var-declaration> ::= KEYWORD(var)
(<identifier-list> COLON <type> SEMICOLON)+

<identifier-list> ::= IDENTIFIER { COMMA IDENTIFIER }

<subprogram-declaration> ::= <procedure-declaration> |
<function-declaration>

<procedure-declaration> ::= KEYWORD(procedure) IDENTIFIER [
<formal-parameter-list>]
SEMICOLON <block> SEMICOLON

<function-declaration> ::= KEYWORD(function) IDENTIFIER [
<formal-parameter-list>]
COLON IDENTIFIER SEMICOLON <block>
SEMICOLON

<formal-parameter-list> ::= LPARENTHESIS <parameter-group>
{ SEMICOLON <parameter-group> }
RPARENTHESIS

<parameter-group> ::= <identifier-list> COLON (<array-type> |
IDENTIFIER)

<compound-statement> ::= KEYWORD(begin) <statement-list>
KEYWORD(end)

<statement-list> ::= <statement> { SEMICOLON <statement> }

<statement> ::= <assignment-statement>
| <if-statement>
| <case-statement>
| <while-statement>
| <repeat-statement>
| <for-statement>
| <procedure/function-call>
| <compound-statement>
| ε

<variable> ::= IDENTIFIER { <component-variable> }

<component-variable> ::= LBRACKET <index-list> RBRACKET
| PERIOD IDENTIFIER

<index-list> ::= (INTEGER_LITERAL | CHAR_LITERAL |
IDENTIFIER)
{ COMMA (INTEGER_LITERAL | CHAR_LITERAL
| IDENTIFIER) }

	<assignment-statement> ::= <variable> ASSIGN_OPERATOR(:=) <expression> <if-statement> ::= KEYWORD(if) <expression> KEYWORD(then) <statement> [KEYWORD(else) <statement>] <case-statement> ::= KEYWORD(case) <expression> KEYWORD(of) <case-block> KEYWORD(end) <case-block> ::= <constant> { COMMA <constant> } COLON <statement> { SEMICOLON [<case-block>] } <while-statement> ::= KEYWORD(while) <expression> KEYWORD(do) <statement> <repeat-statement> ::= KEYWORD(repeat) <statement-list> KEYWORD(until) <expression> <for-statement> ::= KEYWORD(for) IDENTIFIER ASSIGN_OPERATOR(:=) <expression> (KEYWORD(to) | KEYWORD(downto)) <expression> KEYWORD(do) <statement> <procedure/function-call> ::= IDENTIFIER [LPARENTHESIS [<parameter-list>] RPARENTHESIS] <parameter-list> ::= <expression> { COMMA <expression> } <expression> ::= <simple-expression> [<relational-operator> <simple-expression>] <simple-expression> ::= [ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-)] <term> { <additive-operator> <term> } <term> ::= <factor> { <multiplicative-operator> <factor> } <factor> ::= <procedure/function-call> | <variable> | INTEGER_LITERAL | REAL_LITERAL | CHAR_LITERAL | STRING_LITERAL | LPARENTHESIS <expression> RPARENTHESIS | LOGICAL_OPERATOR(not) <factor>

	<relational-operator> ::= RELATIONAL_OPERATOR(==) | RELATIONAL_OPERATOR(<>) | RELATIONAL_OPERATOR(<) | RELATIONAL_OPERATOR(<=) | RELATIONAL_OPERATOR(>) | RELATIONAL_OPERATOR(>=) <additive-operator> ::= ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-) | LOGICAL_OPERATOR(or) <multiplicative-operator> ::= ARITHMETIC_OPERATOR(*) | ARITHMETIC_OPERATOR(/) | ARITHMETIC_OPERATOR(div) | ARITHMETIC_OPERATOR(mod) | LOGICAL_OPERATOR(and)
Start	<program>

5.2.Pembagian Tugas

Tabel 5.3. Pembagian tugas

NIM	**Pembagian tugas**	**Persentase**
13524001	1. Top-level grammar (parseProgram, parseProgramHeader, parseDeclarationPart, parseBlock, parseCompoundStatement, parseStatementList, parseFormalParameterList) 2. Membuat list pembagian tugas implementasi Parser	20%
13524027	1. Membuat kelas Parser yang diturunkan menjadi sejumlah parse-*grammar* dan implementasi fungsi-fungsi traversal dari parse tree. 2. Membuat kelas ParseNode 3. Membuat kelas ParseError dan error	20%

	handling. 4. Membuat landasan teori pada laporan.	
13524029	1. Declarations grammar (parseConstDeclaration, parseConstant, parseTypeDeclaration, parseType, parseArrayType, parseRange, parseEnumerated, parseRecordType, parseFieldList, parseFieldPart, parseVarDeclaration, parseIdentifierList) 2. Membuat bagian perancangan dan implementasi pada laporan	20%
13524089	1. Statements grammar (parseStatement, parseVariable, parseComponentVariable, parseIndexList, parseAssignmentStatement, parseIfStatement, parseCaseStatement, parseCaseBlock, parseWhileStatement, parseRepeatStatement, parseForStatement, parseProcedureFunctionCall) 2. Membuat bagian perancangan dan implementasi pada laporan 3. Membuat bagian pengujian pada laporan 4. Membuat bagian kesimpulan dan saran pada laporan	20%
13524093	1. Expressions (parseExpression, parseSimpleExpression, parseTerm, parseFactor, parseParameterList) 2. Subprograms (parseSubprogramDeclaration, parseProcedureDeclaration,	20%

	parseFunctionDeclaration) 3. Membuat bagian perancangan dan implementasi pada laporan	

5.3.Referensi

Aho, Alfred V, et al. *Compilers : Principles, Techniques, and Tools*. Noida, India, Dorling Kindersley (India) Pvt. Ltd., Licensees Of Pearson Education In South Asia, 2014.